言語処理系
Language Processing System

- ミドルパスにおける最適化
  - 中間語
  - 基本ブロックとその基本的最適化
  - コントロール・フロー・グラフ



早稲田大学理工学術院
情報理工学科
笠原博徳

2026年6月26日配布

コンパイラにおけるフェーズ

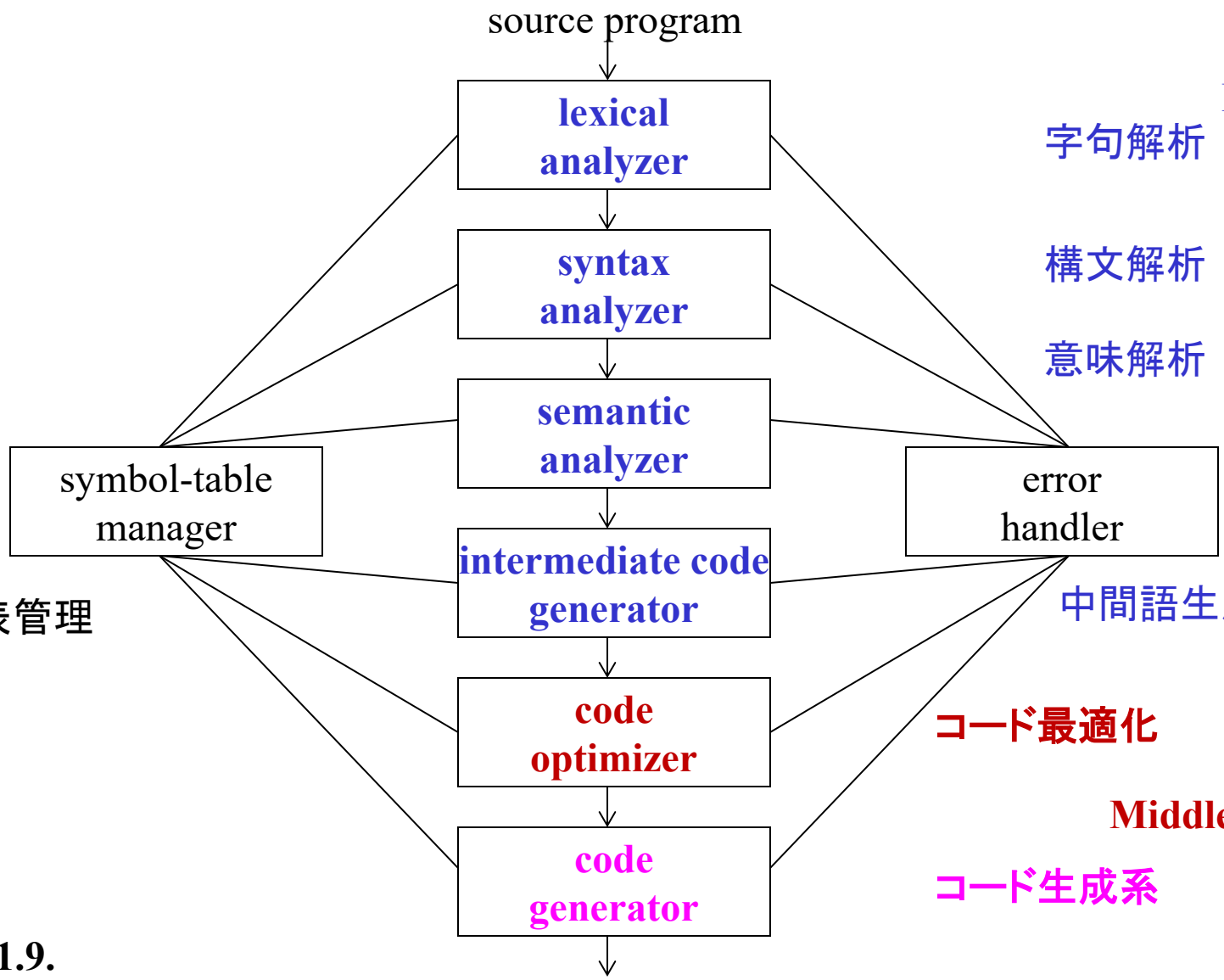


Fig. 1.9.
Phases of a compiler.

ステートメントの翻訳

字句解析(線形解析,走査):ソースプログラムの文字列を左から右にスキャンし、最小の意味単位を構成する文字列である**トークン**にまとめる。
トークン:識別子、代入記号、加算記号、乗算記号、数等

構文解析(階層解析):トークンを読み込んで、文法上の**句**(ひとまとまりの意味のある入れ子状の構造)を作成する。句は解析木で表現される。

意味解析:意味的な間違いがないかとチェックすると共に、タイプチェックをする。**整数と実数の演算等を検査**

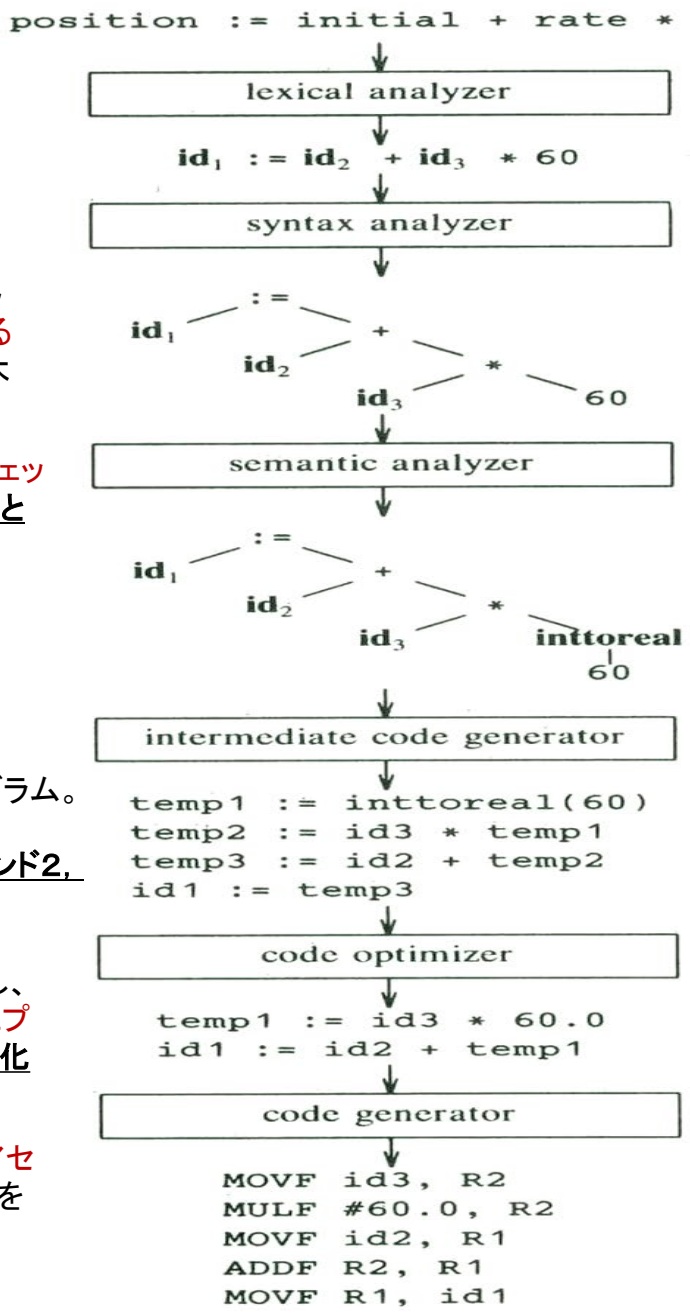
SYMBOL TABLE

position	...
initial	...
rate	...

中間語: 一種の抽象機械に対するプログラム。
3アドレスコード(四つ組)
演算子、ソースオペランド1、ソースオペランド2、デスティネーションオペランド

最適化:制御及びデータの流れ等を解析し、実行速度の高いコードを作成できるようにプログラムを変換する。**並列化、メモリ最適化**

コード生成:ターゲットとなるマシン用のアセンブリコード、リロケータブルマシンコードを生成する。 **レジスタ最適**化。



字句解析

構文解析

意味解析

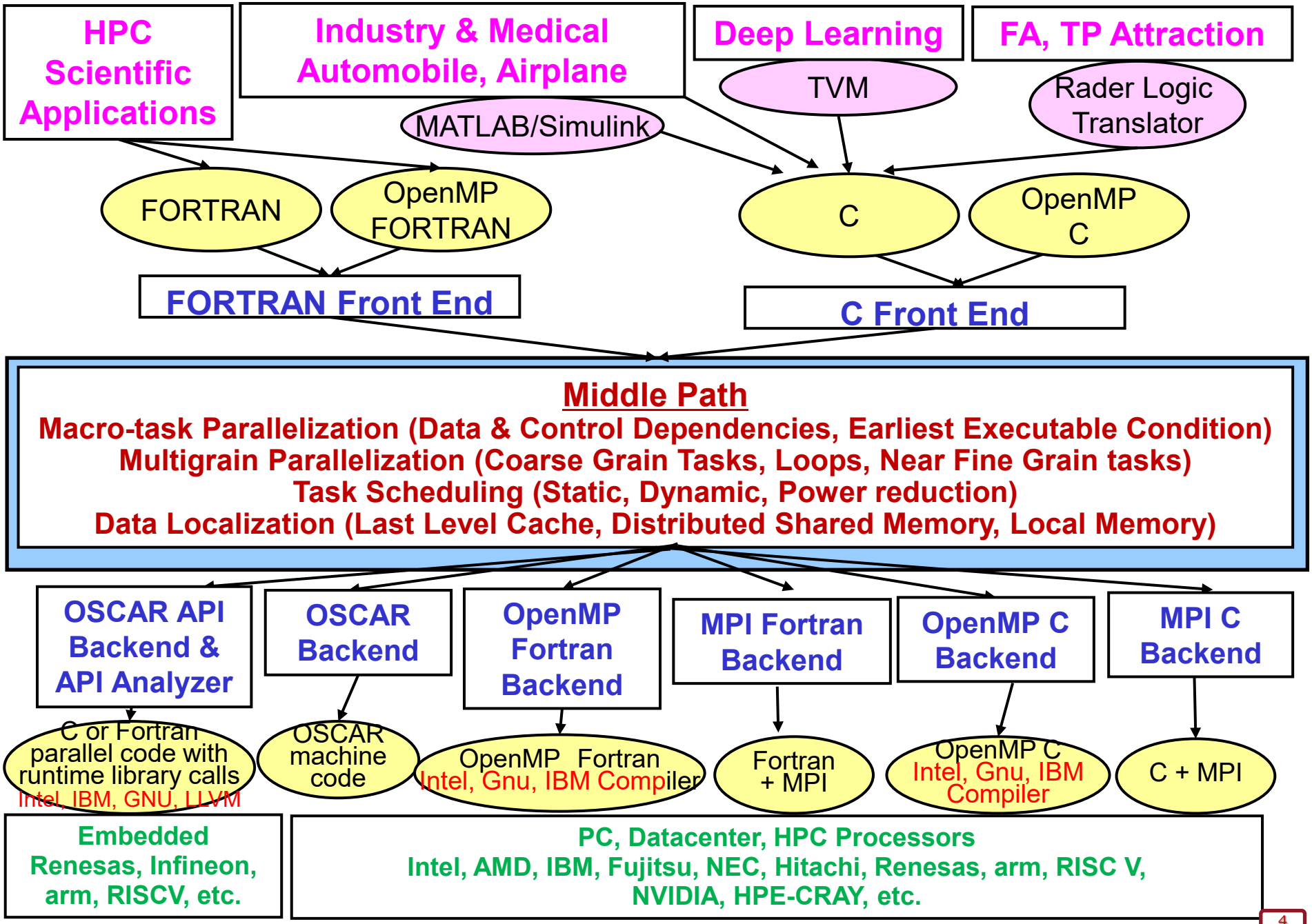
中間語生成

コード最適化

コード生成系

Fig. 1.10.
Translation of a statement.

フロントエンド・ミドルパス・バックエンドの関係：中間語の利用により各種言語・マシンに対応



ccNUMA

- cache coherent Non-Uniform Memory Access

– メモリは、物理的には、システム全体に分散

Memory is physically distributed.

– メモリは、システム全体でアドレスを共有

Memory addresses are shared by the all processors.

– ローカルメモリの参照は、リモートメモリへの参照よりも高速
(Non-Uniform Memory Access = NUMA)

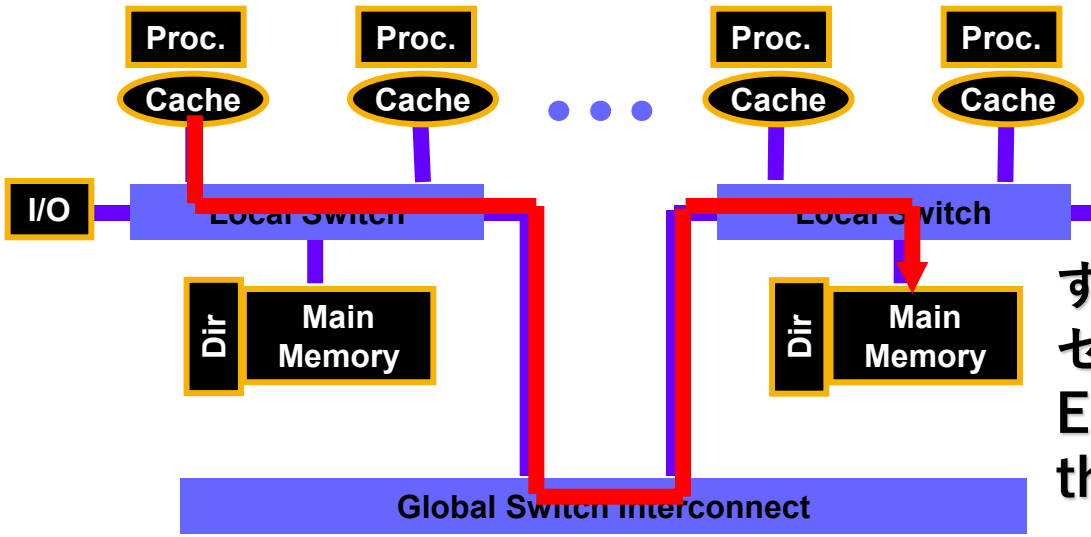
Access latency to local memory is smaller than
access latency to remote memory



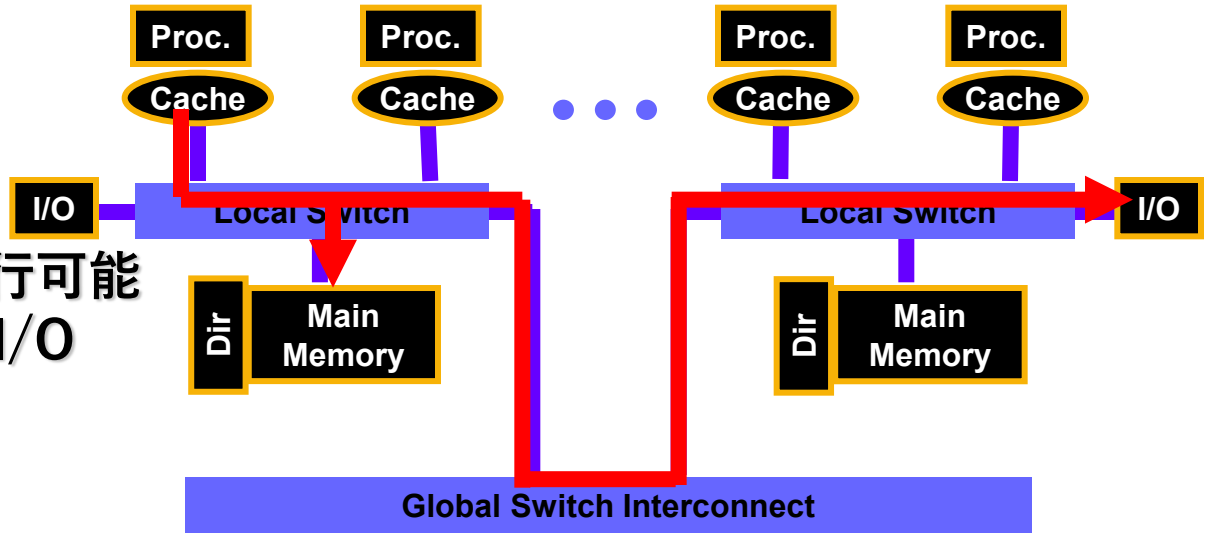
Prof. John L. Hennessy
Dash Project（1992）, SGI (Silicon Graphics International)より製品化,
Origin 2000(1996, Based on MIPS R10000 processor）,
Altix 3000 (2003, Based on Intel Itanium 2 processor）NASA 惑星シミュレータ
スタンフォード大前学長, Alphabet会長（Googleの親会社）
ACMチューリング賞, IEEE Medal of Honor, IEEE Seymour Cray Award
Invited Talk: H. Kasahara, "OSCAR Fortran Multigrain Compiler", Stanford University, Hosted by Professor John L. Hennessy and Professor Monica Lam, May. 15. 1995.

SGI ccNUMA



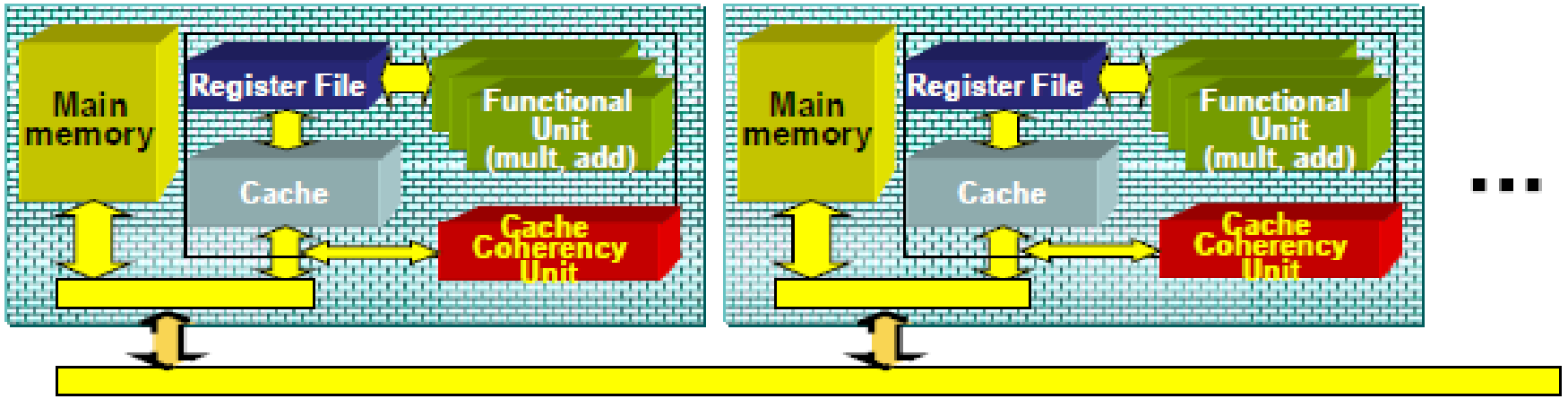


すべてのメモリに直接アクセスが可能
Every Proc. can access the all memory directly.



すべてのI/Oを直接に実行可能
Every Proc. can make I/O processing directly.

Distributed Shared Memory Architecture (ccNUMA)
分散共有メモリ型アーキテクチャ（ccNUMA）



- Each processor has Local Memory.
- All memory is logically shared.
- Local memory access is faster than remote memory access.

– プロセッサはそれぞれローカルメモリを持つ
– 全てのメモリで論理的に“共有メモリ”を構成
– Non-Uniform Memory Access : cache-coherent NUMA (ccNUMA)
  • ローカルメモリへのアクセスはリモートメモリへのアクセスより速い
– 共有メモリ型アーキテクチャのように自動並列化が可能
– 分散メモリ型アーキテクチャのように拡張性がある

- Programing is easy like shared memory architectures.
- Scalable like distributed memory architectures.

sgi INNOVATION FOR RESULTS

D. Lenoski, J. Laudon, K Gharachorloo, A. Gupta, J. Hennessy, “The directory-based cache coherence protocol for the DASH multiprocessor,” ACM SIGARCH Computer Architecture News 18 (2SI), 148-159, 1990

H. Kasahara, "OSCAR Fortran Multigrain Compiler", Stanford University, Hosted by Professor John L. Hennessy and Professor Monica Lam, May. 15. 1995.

年200億個以上のプロセッサが生産

マルチコアプロセッサ：スマホ,タブレット,IoTデバイス,自動車制御,サーバ,スパコン等

例：ARM,IBM Power,Renesas RH850, Infineon,SPARC,RISC V



https://www.apple.com/jp/shop/buy-iphone

64-bit
iPhone 13
2021



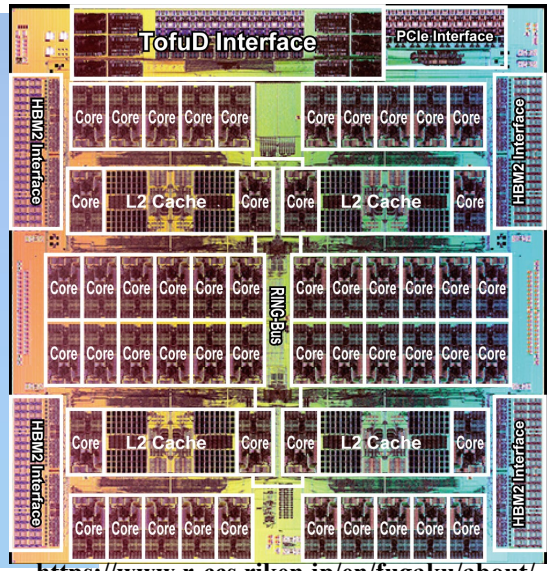
Launched September 14, 2021
Designed by Apple Inc.
Common manufacturer(s)： TSMC
Max. CPU clock rate to 3.23 GHz in iPhone 13 Pro
Technology node: 5 nm
6 Cores: 2 “Avalanche“高性能コア & 4 ”Blizzard”省エネコア
Instruction set: A64, Transistors: 15 billion (15億個)
GPU(s): Apple-designed 5 core GPU in iPhone 13

https://en.wikipedia.org/wiki/Apple_A15

理研富岳スーパーコンピュータ 2020年6月から2021年11月まで世界No.1



https://fugaku100kei.jp/fugaku/



https://www.r-ccs.riken.jp/en/fugaku/about/

RIKEN Center for Computational Science, Fujitsu（arm based processor）
Cores:7,299,072； Memory:4,866,048GB；
Processor:A64FX 48Cores, 2.2GHz
Interconnect: Tofu interconnect D
Linpack (Rmax)415,530 TFlop/s;
Theoretical Peak (Rpeak): 513PFLOPS
HPCG [TFlop/s]13,366.4; Power: 28.3MW

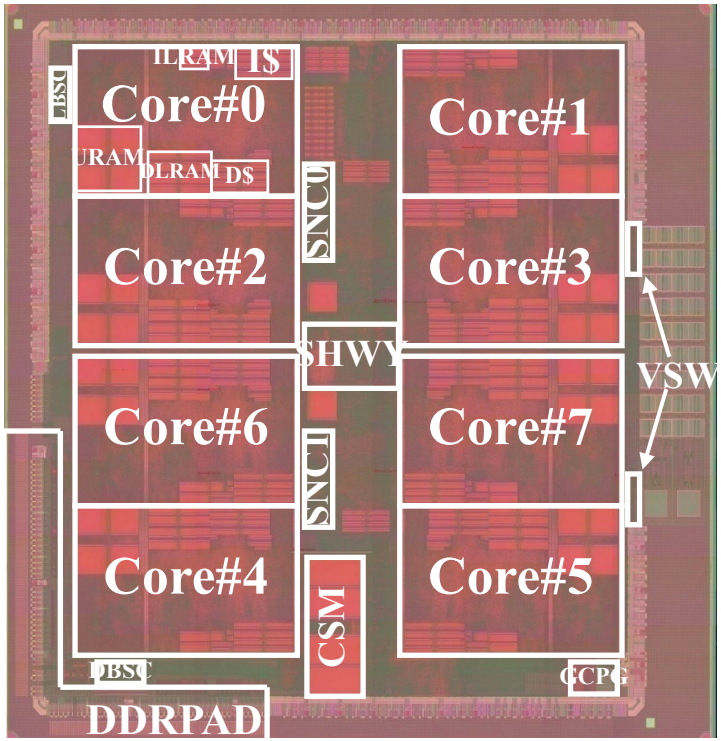
48コア/チップ, 2.2GHz, 7 nm FinFET,
約7百30万コア, 28MW
理論最高性能:51京回浮動小数点演算/秒,
2020年6月時点

https://japanese.engadget.com/arm-super-computer-fugaku-top-500-034015910.html

ムーアの法則の終焉

ムーアの法則(Moore's law): インテル創業者の一人であるゴードン・ムーア(Gordon E. Moore: IEEE Computer Pioneer Award)が、1965年の論文で提唱した「半導体の集積率は18か月で2倍になる」という経験則。

コンピュータの高性能化と低消費電力化にはマルチコアが必須



IEEE ISSCC08: Paper No. 4.5, M.ITO, … and H. Kasahara, "An 8640 MIPS SoC with Independent Power-off Control of 8 CPUs and 8 RAMs by an Automatic Parallelizing Compiler"

$$\text{Power} \propto \text{Frequency} * \text{Voltage}^2$$

⇒ (Voltage ∝ Frequency)

$$\text{Power} \propto \text{Frequency}^3$$

周波数 Frequency を1/4にすると (Ex. 4GHz→1GHz),

消費電力は 1/64に削減

性能は 1/4に低下 .

<マルチコア>

8cores をチップに集積すると,

電力は 依然1/8で性能 は2 倍向上

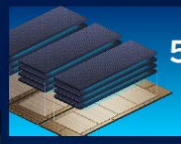
https://www.intel.com/content/www/us/en/content-details/921464/data-center-business-strategy-computex-2026.html

Data Center Business Strategy

Kevork Kechichian - Executive Vice President and General Manager, DCG

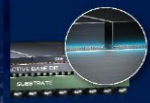
Intel® Xeon® 6+

First 18A CPU in the Data Center



576MB LLC

96 lanes PCIe Gen 5

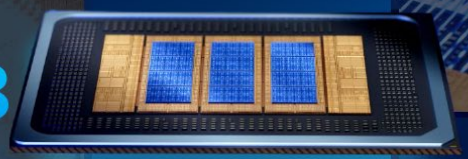


Foveros Direct 3D

12ch DDR5 8000



288 Next-gen E-cores



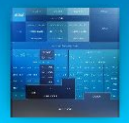
intel 18A

Security built in

Intel SGX
Intel TDX

330–450W TDP

Next-gen Darkmont E-cores







Intel Application Energy Telemetry

64 lanes CXL Compute Express Link



1-2 Socket

intel XEON

Launched June 1ST 2026

18A（1.8nmクラス）
14A（1.4nmクラス）

技術要素	Intel 18A (2025年量産)	Intel 14A (2026年リスク生産/2027年量産)
トランジスタ	RibbonFET (GAA構造)	RibbonFET (さらに最適化)
給電方式	PowerVia (裏面給電)	PowerDirect (第2世代裏面給電)
露光装置	EUV (標準)	High-NA EUV (世界初導入)
主な特徴	5N4Y計画の集大成。PPAを最大化。	圧倒的な密度向上。High-NAによる微細化。

https://tm-co.co.jp/glossary/intel%E3%81%AE%E8%A3%BD%E9%80%A0%E3%83%97%E3%83%AD%E3%82%BB%E3%82%B9%EF%BC%8818a-14a%EF%BC%89

Intel® Xeon® 6+

Next-level server CPU density & efficiency, built on Intel 18A



Exceptional performance

Up to **288 E-cores**
Highest core density

Up to **8000 MT/s DDR5**
Improved memory

Up to **576 MB**
Enhanced low latency LLC

Efficient compute

Intel 18A
PowerVia & RibbonFET

Market leading
Rack density

Intel® AET
Intel® Application Energy Telemetry (Intel® AET)

Trusted and secure

Confidential compute
Most comprehensive portfolio

Intel® SGX and Intel® TDX

Crypto algorithm acceleration
for SHA-512, SM3 and SM4



OpenMP

An Application Program Interface (API) that may be used to explicitly direct multi-threaded, shared memory parallelism

- **Comprised of three primary API components:**
  - **Compiler Directives**
  - **Runtime Library Routines**
  - **Environment Variables**

Portable: The API is specified for C/C++ and Fortran

Multiple platforms have been implemented including most Unix platforms and Windows NT

Standardized:

Jointly defined and endorsed by a group of major computer hardware and software vendors

Prof. David Kuck

Univ. Illinois at Urbana-Champaign
Prof. Emeritus
Founder of Kuck & Associates

- ILLIAC IV (Member)
- Parallelizing Compiler
  - Data dependence analysis
  - Automatic vectorization
  - Automatic loop parallelization
  - Loop restructuring
  - Cedar supercomputer (Alliant FX8)
  - OpenMP

Intel Senior Fellow (Past CTO):

- Intel Parallel Tuning Tools & Debuggers

- IEEE Fran Allen Medal
- National Academy of Engineering Member
- IEEE ACM Eckert-Mauchly Award
- IEEE Charles Babbage Award
- ACM/IEEE Ken Kennedy Award
- IEEE Computer Pioneer Award
- Okawa Prize

Prof. David J Kuck (Univ. Illinois, Intel)
IEEE Computer Pioneer Award 2011
Ms. Diane B. Greene (VMware Cofounder & CEO)
IEEE Computer Society Computer Entrepreneur Award 2011





2018.03.08 WasedaUniv. Symposium on Future of High Performance Green Computing 2018 (HPGC2018)



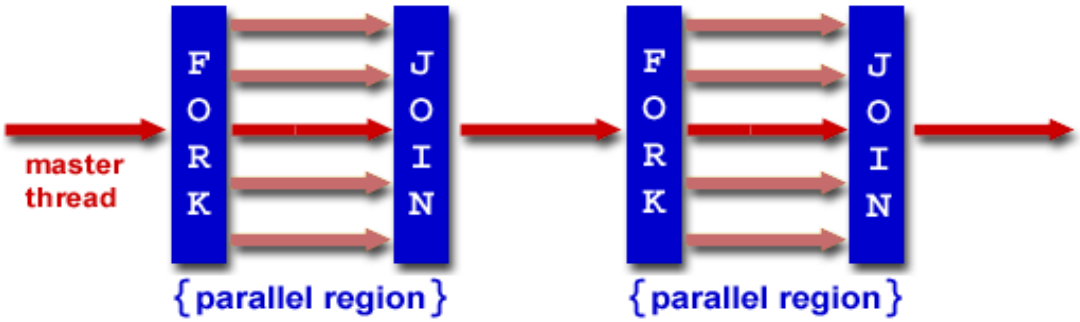




OpenMP Programming Model

•**Thread Based Parallelism:**

•**Explicit Parallelism:**



•**Fork - Join Model:**

•**Compiler Directive Based:**

Virtually all of OpenMP parallelism is specified through the use of compiler directives which are imbedded in C/C++ or Fortran source code.

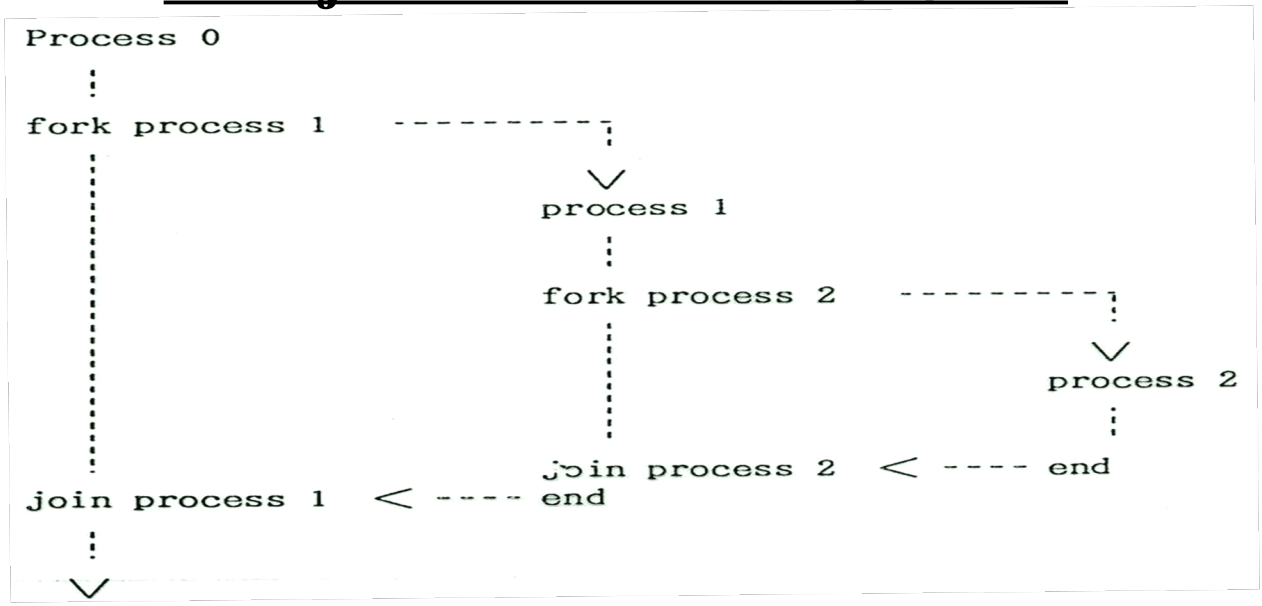
•Nested Parallelism Support:

The API provides for the placement of parallel constructs inside of other parallel constructs.

•Dynamic Threads:

The API provides for dynamically altering the number of threads which may used to execute different parallel regions.

fork join による並列性記述



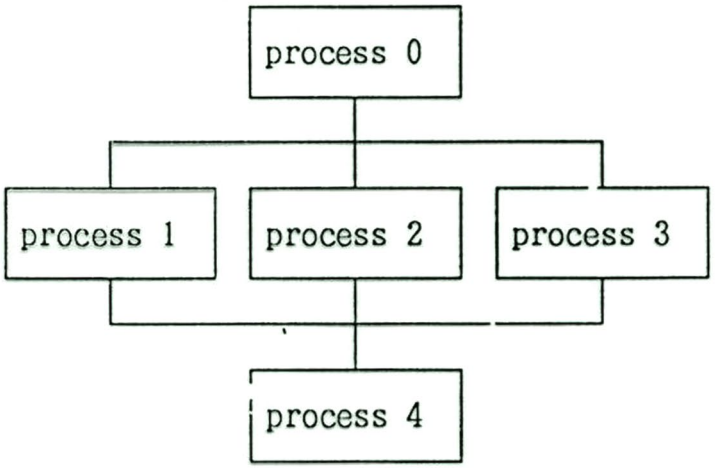
cobegin coend による並列性記述

```
begin

    process 0;

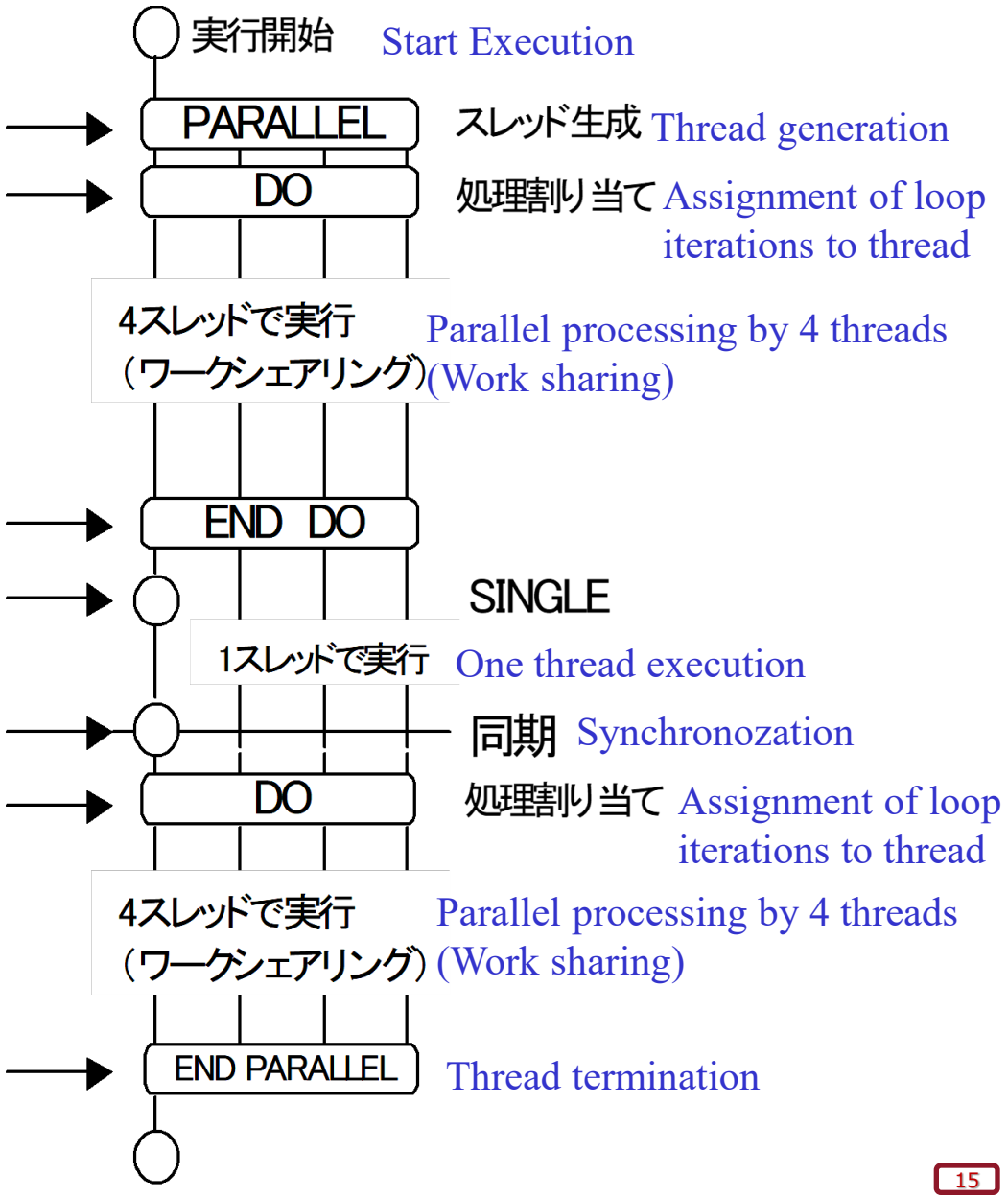
cobegin process 1; process 2; process 3; coend

    process 4;
end
```



OpenMPプログラム記述例

```
> set OMP_NUM_THREADS=4
> test.exe
!$OMP PARALLEL PRIVATE(temp)
!$OMP DO
        do I = 1, 10
                temp = p * 10.0
                a(I) = sqrt(temp)
        end do
!$OMP END DO nowait
!$OMP SINGLE
        sum = 0.0
!$OMP END SINGLE
!$OMP DO REDUCTION(+:sum)
        do I = 1, 10
          sum = sum + a(I)
        end do
!$ OMP END PARALLEL
```



1. パラレルリージョン構造 Parallel region

PARALLEL ディレクティブ

```
!$OMP PARALLEL [clause [[,] clause…]
block
!$OMP END PARALLEL
```

clause: PRIVATE, SHARED, DEFAULT, FIRSTPRIVATE, LASTPRIVATE, REDUCTION, IF, COPYIN

2. ワークシェアリング構造 Work sharing

DO ディレクティブ

```
!$OMP DO [clause [[,] clause…]
do loop
[!$OMP END DO [NOWAIT]]
```

clause: PRIVATE, REDUCTION, SCHEDULE, FIRSTPRIVATE, LASTPRIVATE, ORDERD

SECTION ディレクティブ

```
!$OMP SECTIONS [clause [[,] clause…]
[!$OMP SECTION]
block
[!$OMP SECTION
……]
!$OMP END SECTIONS [NOWAIT]
```

clause: PRIVATE, FIRSTPRIVATE, LASTPRIVATE, REDUCTION

SINGLE ディレクティブ

```
!$OMP SINGLE [clause [[,] clause…]
block
!$OMP END SINGLE [NOWAIT]
```

clause: PRIVATE, FIRSTPRIVATE

3. 複合構造

PARALLEL DO ディレクティブ

```
!$OMP PARALLEL DO [clause [[,] clause…]
do loop
[!$OMP END PARALLEL DO]
```

PARALLEL SECTIONS ディレクティブ

```
!$OMP PARALLEL SECTIONS [clause [[,] clause…]
[!$OMP SECTION]
block
[!$OMP SECTION]
……]
!$OMP END PARALLEL SECTIONS
```

4. 同期構造 Synchronization

MASTER ディレクティブ

```
!$OMP MASTER
block
!$OMP END MASTER
```

CRITICAL ディレクティブ

```
!$OMP CRITICAL [name]
block
!$OMP END CRITICAL [name]
```

BARRIER ディレクティブ

```
!$OMP BARRIER
```

ATOMIC ディレクティブ

```
!$OMP ATOMIC
x = x operator exper などの式
```

FLUSH ディレクティブ

```
!$OMP FLUSH [(list)]
```

ORDERED ディレクティブ

```
!$OMP ORDERED
block
!$OMP END ORDERED
```

5. データ環境構造

THREADPRIVATE ディレクティブ

```
!$OMP THREADPRIVATE(/cb,[,/cb]…)
```

Data Assignment (Problems in a case without the first-touch)
データ配置 (first-touch) (2/3)

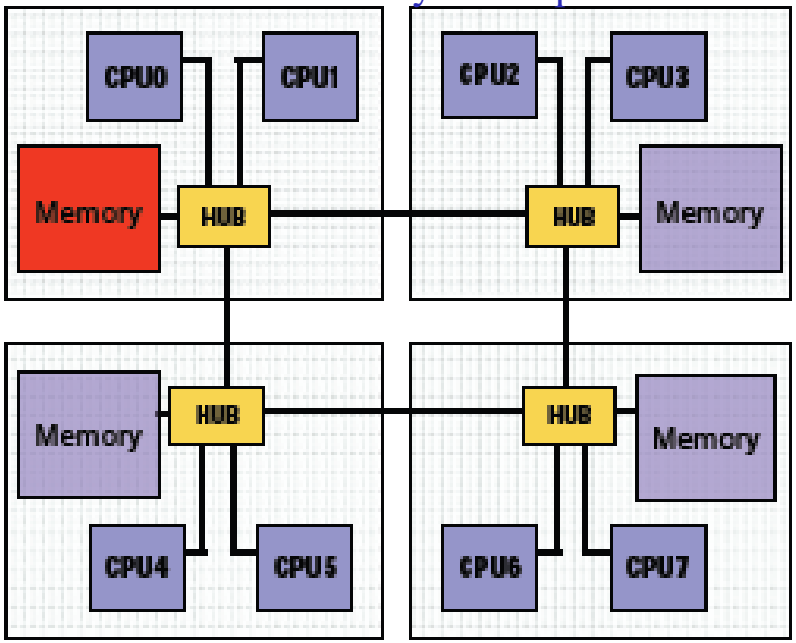
If a single processor initializes the all data, the all data is allocated to local memory of the processor.

- "first-touch"ポリシーにおけるシングルプロセッサでの初期化
  - すべてのデータはひとつのノード上に
  - そのノードへのアクセスがボトルネック

The processor will be a bottleneck because the all processors access the memory of the processor.

```
        do  i=1, n
           A(i) = 0.
           B(i) = i/2
           C(i) = i/3
           D(i) = i/7
        enddo
!$omp parallel do private(i) shared(A,B,C,D)
        do  i=1, n
            A(i) = B(i) + C(i) + D(i)
        enddo
```

The first touch by a sequential loop.
逐次実行による
ファーストタッチ



Data Assignment (Merits by the First-touch)
データ配置 (first-touch) (3/3)

- “first-touch”ポリシーにおける複数のプロセッサによる並列化されたループでの初期化

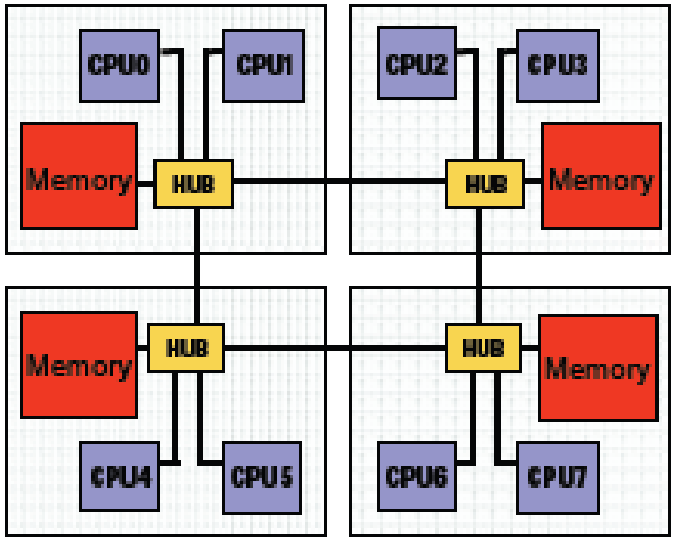
  If the all data is initialized by a parallel loop, the data is allocated to each processor, remote memory access will be minimized..

  - データは最適に配置される
  - それぞれのプロセッサはローカルにデータを配置
  - ノード間のデータ交換は最小

```
!$omp parallel do private(i)shared(A,B,C,D)
        do  i=1, n
          A(i) = 0.
          B(i) = i/2
          C(i) = i/3
          D(i) = i/7
        enddo
!$omp parallel do private(i) shared(A,B,C,D)
        do  i=1, n
        A(i) = B(i) + C(i) + D(i)
        enddo
```

The first touch by a parallel loop.

並列実行によるファーストタッチ



- 計算で行われるデータアクセスのように初期化することにより、性能向上が図れます。

  By the first-touching the data as the following parallel loops, the application performance will be improved significantly.

スーパーテクニカルサーバ「SR16000モデルVM1」

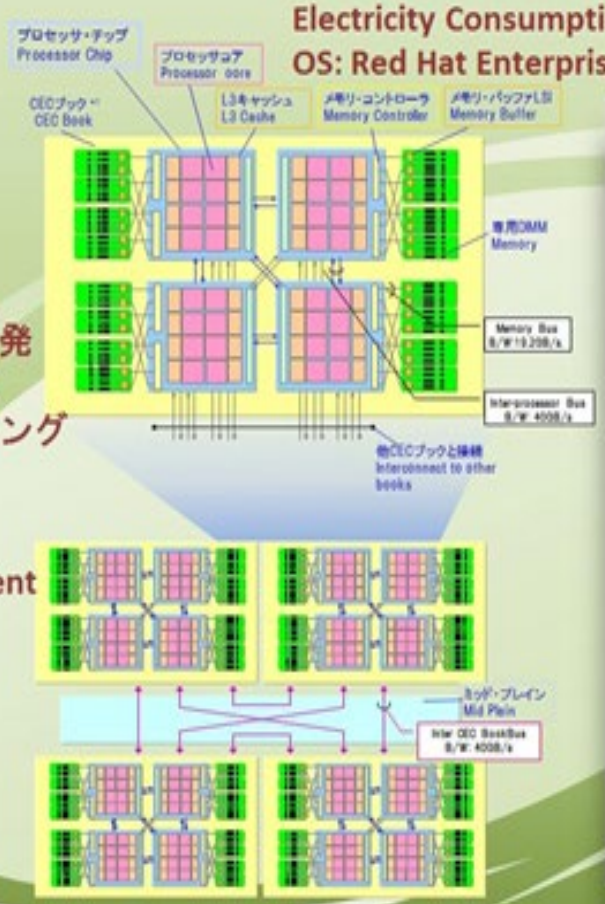
Kasahara & Kimura Laboratory

Green Computing
Research Center Project

CPU : POWER7（4.0GHz）× 128Core
Memory : 1024GB
Peak Perfromance:4.1TFLOPS
Electricity Consumption : 256.8MFLOPS/W
OS: Red Hat Enterprise Linux 6

【用途】
・並列化コンパイラ開発
・低消費電力メニーコア開発
・屋上太陽光発電を用いた
グリーンコンピューティング
・災害シミュレーション

【Purpose】
・Parallelizing compiler development
・Development of low power
manycore processors
・Green Computing
by rooftop solar power
・Disaster Simulation

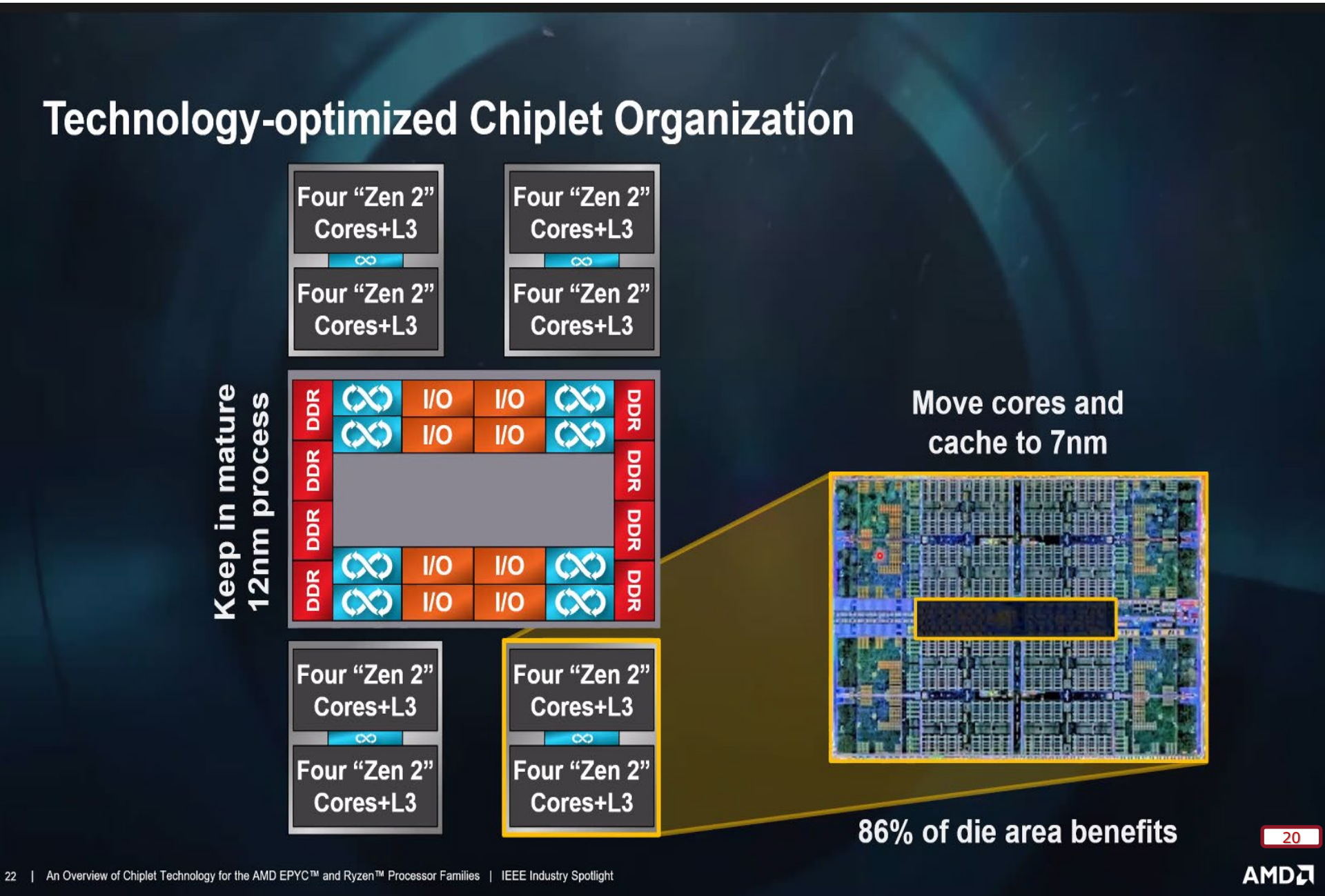


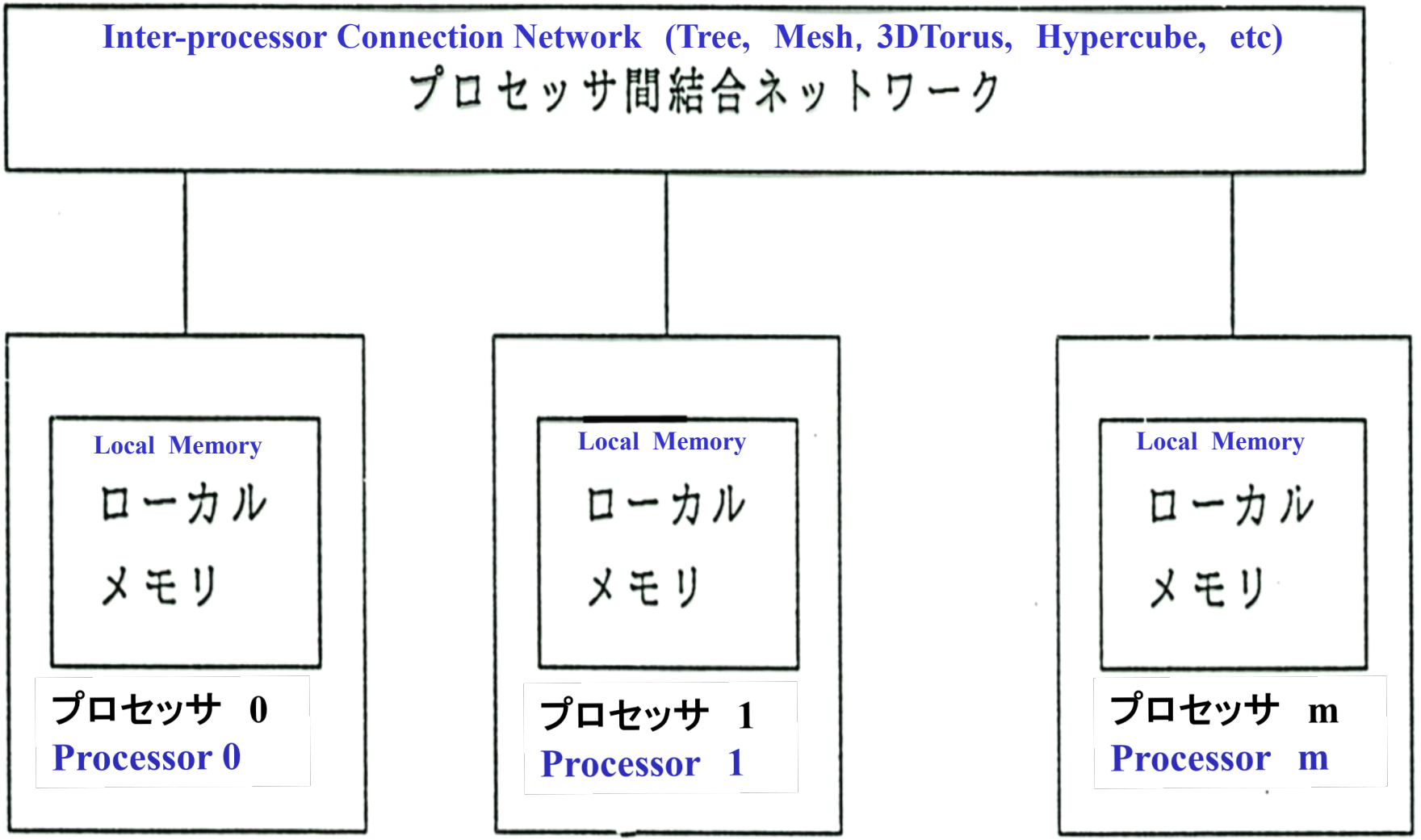


HITACHI

WASEDA Univ.

AMD最新チップの例：64コア　Zen2　チップレット構成プロセッサ
プロセッサ・チップレット7nm、　メモリ・チップレット　12nm





分散メモリ型(疎結合型/メッセージパシング型)マルチプロセッサ・システム

Distributed Memory (Loosely Connected/Message Passing) Multiprocessor System

TOP10 System - November 2023

https://www.top500.org/ Nov. 2023

Rmax and **Rpeak** values are in PFlop/s. For more details about other fields, check the TOP500 description.

Rpeak values are calculated using the advertised clock rate of the CPU. For the efficiency of the systems you should take into account the Turbo CPU clock rate where it applies.

Rank	System	Cores	Rmax (PFlop/s)	Rpeak (PFlop/s)	Power (kW)
1	**Frontier** - HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, **HPE** DOE/SC/Oak Ridge National Laboratory **United States**	8,699,904	1,194.00	1,679.82	22,703
2	**Aurora** - HPE Cray EX - Intel Exascale Compute Blade, Xeon CPU Max 9470 52C 2.4GHz, Intel Data Center GPU Max, Slingshot-11, **Intel** DOE/SC/Argonne National Laboratory **United States**	4,742,808	585.34	1,059.33	24,687
3	**Eagle** - Microsoft NDv5, Xeon Platinum 8480C 48C 2GHz, NVIDIA H100, NVIDIA Infiniband NDR, **Microsoft** Microsoft Azure **United States**	1,123,200	561.20	846.84	
4	**Supercomputer Fugaku** - Supercomputer Fugaku, A64FX 48C 2.2GHz, Tofu interconnect D, **Fujitsu** RIKEN Center for Computational Science **Japan**	7,630,848	442.01	537.21	29,899
5	**LUMI** - HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, **HPE** EuroHPC/CSC **Finland**	2,752,704	379.70	531.51	7,107





No. 1 June 2022 **873万プロセッサ, 21MW**

Frontier - HPE Cray EX235a, 8,730,112 total cores, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X accelerators, HPE Slingshot-11 interconnect **168.5京回演算/秒**

Oak Ridge National Laboratory (ORNL) , USA

Rmax: 1.102 (ExaFlop/s), Rpeak 1.685 (ExaFlop/s)

Power:21.1 MW, Efficiency: 52.23 gigaflops/W

https://www.computer.org/profiles/william-gropp



William D. Gropp

2022 IEEE Computer Society President

MPI （Message Passing Interface)

Biography

Dr. William "Bill" Gropp is Director of the National Center for Supercomputing Applications and holds the Thomas M. Siebel Chair in the Department of Computer Science at the University of Illinois at Urbana-Champaign. He received his Ph.D. in Computer Science from Stanford University in 1982. He was on the faculty of Computer Science at Yale from 1982-1990. From 1990-2007, he was a member of the Mathematics and Computer Science Division of Argonne National Laboratory, including serving as associate division director.

Gropp has served the Computer Society as Vice President for Technical and Conference Activities since 2019 and has been a member of the Board of Governors since 2017. As chair of Technical Program (2009), Finance (2011), and as General Chair (2013) of the SC conference, which at over 12,000 attendees is the largest conference co-sponsored by the Computer Society, Gropp has experience leading volunteers (almost 600 for SC) and running financially successful technical conferences.

He played a major role in the development of the MPI message-passing standard, used around the world to program high performance computing systems. He is co-author of MPICH, a widely used implementation of MPI, and has written many books and papers on MPI. He is also one of the designers of the PETSc parallel numerical library and has developed parallel algorithms for the solution of linear and nonlinear equations. Dr. Gropp co-chaired the National Academy's Committee on Future Directions for NSF Advanced Computing Infrastructure to Support U.S. Science.

Gropp is a Fellow of AAAS, ACM, IEEE, and SIAM and a member of the National Academy of Engineering. He received the Sidney Fernbach Award from the Computer Society in 2008 and the ACM/IEEE-CS Ken Kennedy Award "for highly influential contributions to the programmability of high-performance parallel and distributed computers, and extraordinary service to the profession" in 2016.

Recent Volunteer Positions

Vice President, Technical & Conference Activities (2020 ExCom)
2020 Board of Governors
2017–2019 Board of Governors
Learn more about volunteering

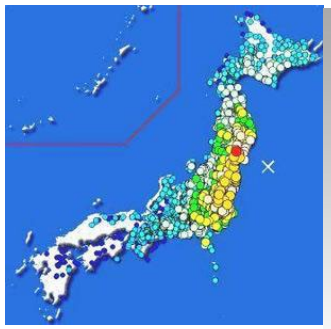
Awards

2016 Ken Kennedy Award
"For highly influential contributions to the programmability of high-performance parallel and distributed computers, and extraordinary service to the profession."
Learn more about the Ken Kennedy Award

2008 Sidney Fernbach Award
"For outstanding contributions to the development of domain decomposition algorithms, scalable tools for the parallel numerical solution of PDEs, and the dominant HPC communications interface."
Learn more about the Sidney Fernbach Award

Parallel Software is important for scalable performance of multicore (LCPC2015)

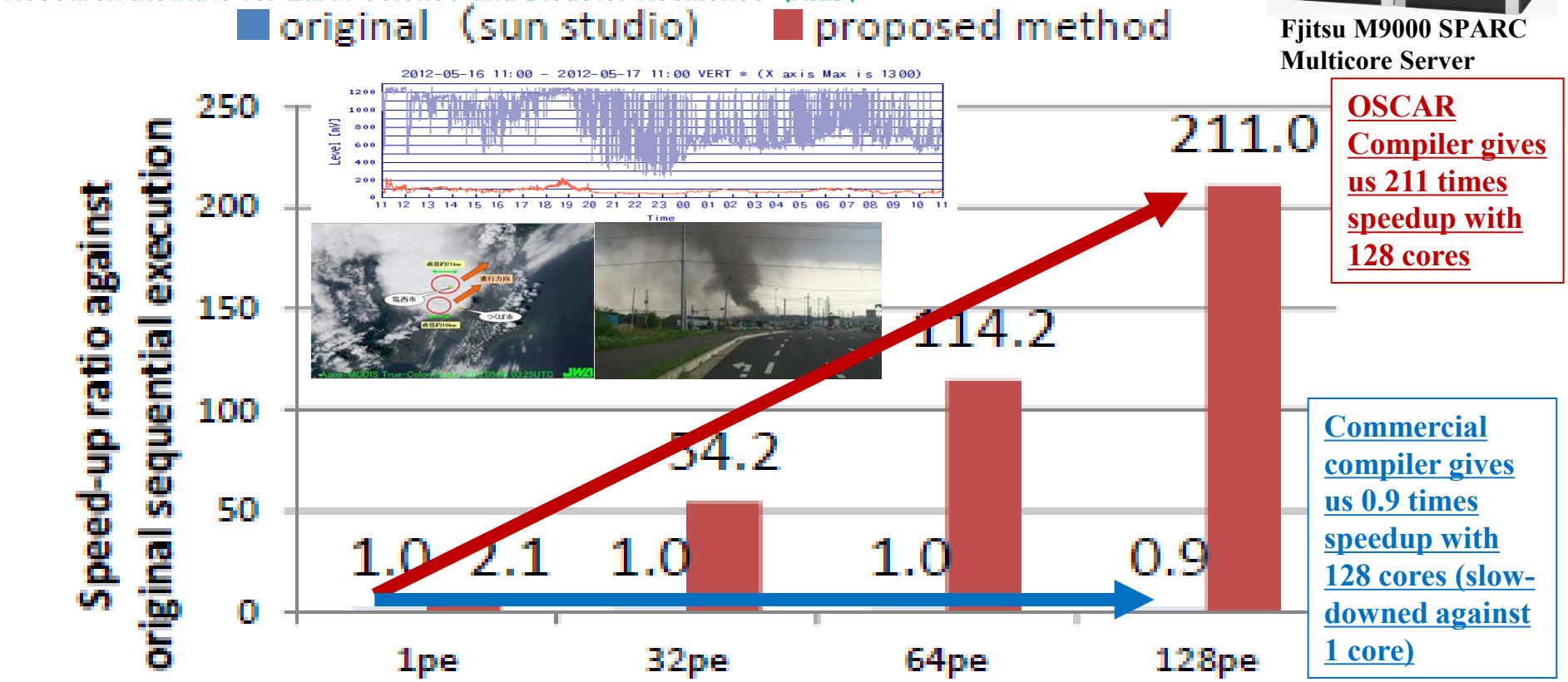
- Just more cores don't give us speedup
- Development cost and period of parallel software are getting a bottleneck of development of embedded systems, eg. IoT, Automobile





Fjitsu M9000 SPARC Multicore Server

Earthquake wave propagation simulation GMS developed by National Research Institute for Earth Science and Disaster Resilience (NIED)



OSCAR Compiler gives us 211 times speedup with 128 cores

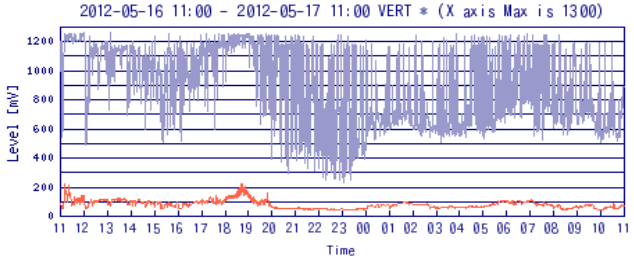
Commercial compiler gives us 0.9 times speedup with 128 cores (slow-downed against 1 core)

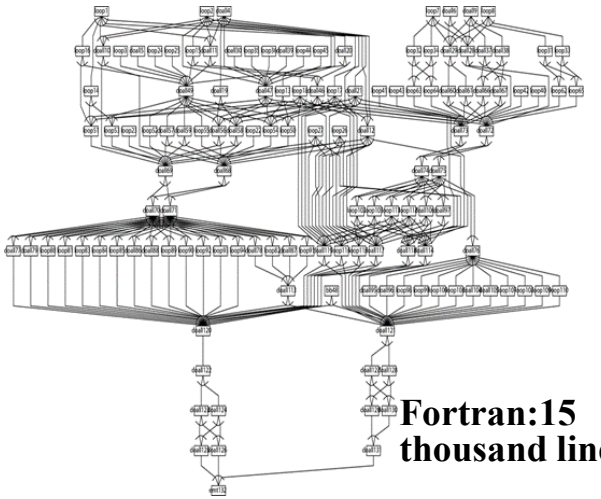
- Automatic parallelizing compiler available on the market gave us no speedup against execution time on 1 core on 64 cores
  - Execution time with 128 cores was slower than 1 core (0.9 times speedup)
- Advanced OSCAR parallelizing compiler gave us 211 times speedup with 128cores against execution time with 1 core using commercial compiler
  - OSCAR compiler gave us 2.1 times speedup on 1 core against commercial compiler by global cache optimization

110 Times Speedup against the Sequential Processing for GMS Earthquake Wave Propagation Simulation on Hitachi SR16000

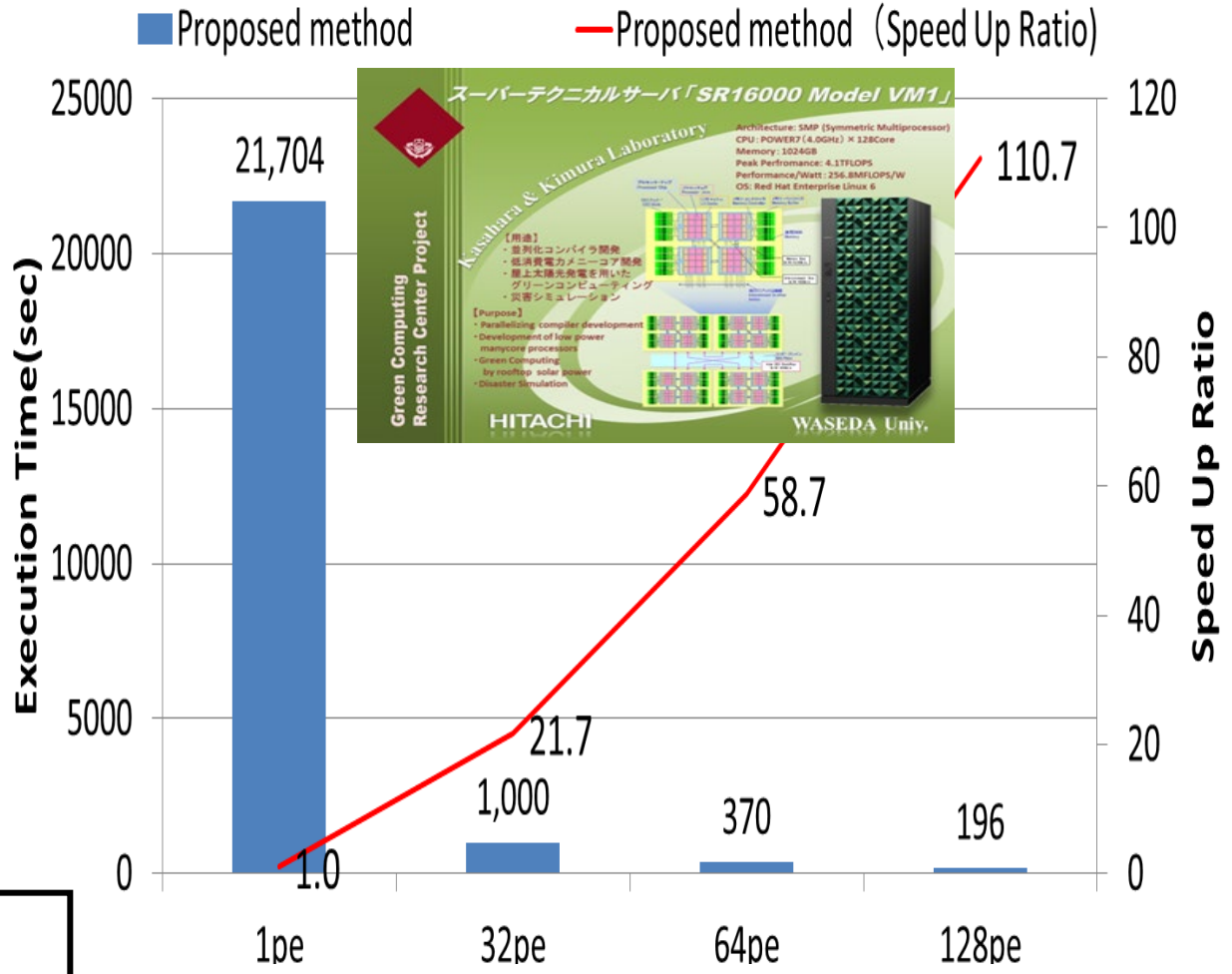
（Power7 Based 128 Core Linux SMP） (LCPC2015)







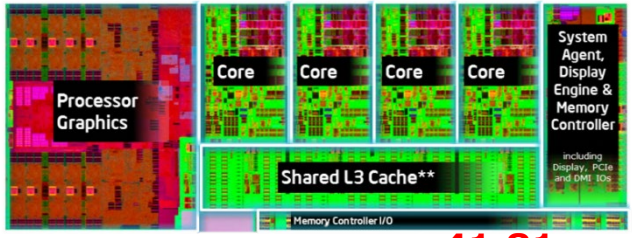
Fortran:15 thousand lines



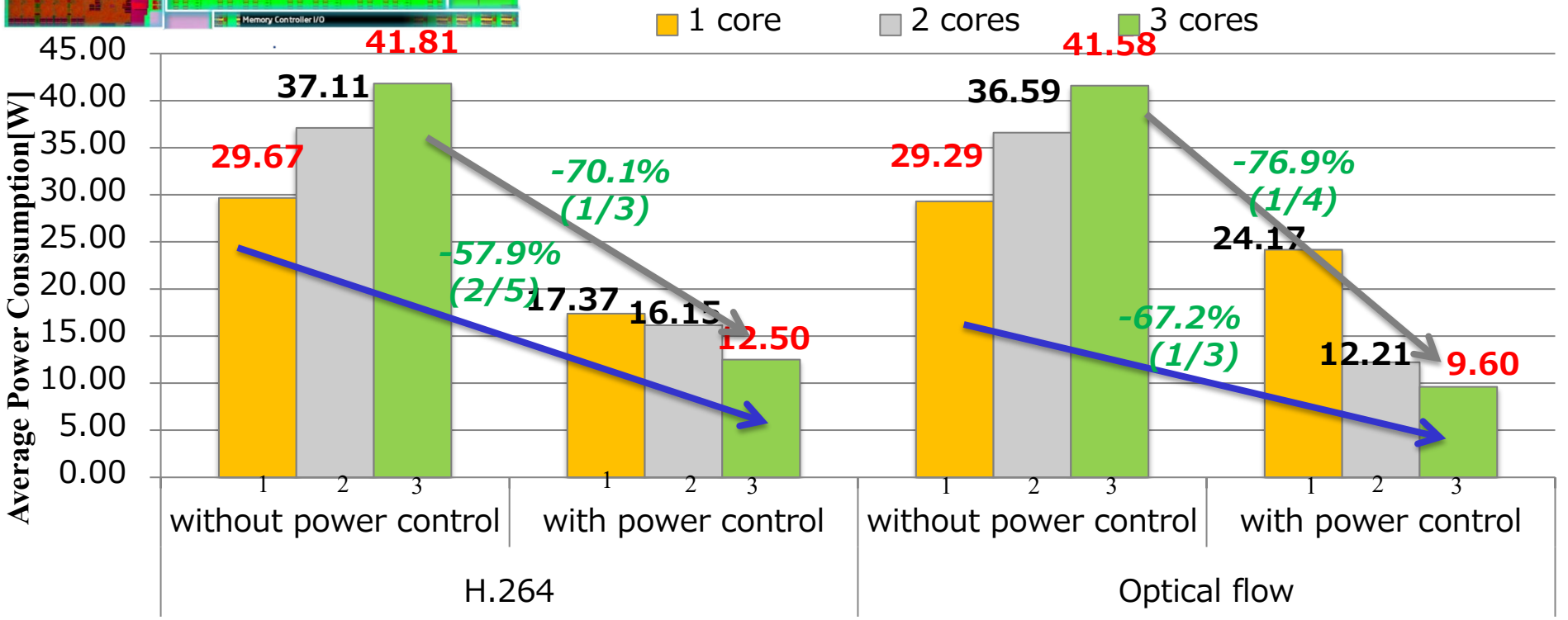
First touch for distributed shared memory and cache optimization over loops are important for scalable speedup

Intel Haswell上での電力削減

H.264 decoder & Optical Flow （3コア使用）



H81M-A，Intel Core i7 4770k
Quad core，3.5GHz～0.8GHz



3PE電力制御なしと3PE電力制御ありで電力を1/3～1/4に削減
1PE電力制御なしと3PE電力制御ありで電力を2/5～1/3に削減

Multicore Program Development Using OSCAR API V2.0

Sequential Application Program in Fortran or C
(Consumer Electronics, Automobiles, Medical, Scientific computation, etc.)

Homogeneous

Hetero

Manual parallelization / power reduction

Accelerator Compiler/ User
Add "hint" directives before a loop or a function to specify it is executable by the accelerator with how many clocks

Waseda OSCAR Parallelizing Compiler
- Coarse grain task parallelization
- Data Localization
- DMAC data transfer
- Power reduction using DVFS, Clock/ Power gating

OSCAR API for Homogeneous and/or Heterogeneous Multicores and manycores
Directives for thread generation, memory, data transfer using DMA, power managements

Parallelized API F or C program

Proc0
Code with directives
Thread 0

Proc1
Code with directives
Thread 1

P-thread

Accelerator 1
Code

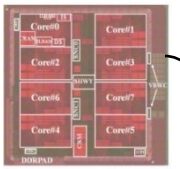
Accelerator 2
Code

Low Power Homogeneous Multicore Code Generation
API Analyzer | Existing sequential compiler

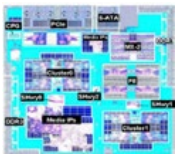
Low Power Heterogeneous Multicore Code Generation
API Analyzer (Available from Waseda) | Existing sequential compiler

Server Code Generation
OpenMP Compiler

Generation of parallel machine codes using sequential compilers



Homegeneous Multicore s from Vendor A (SMP servers)



Heterogeneous Multicores from Vendor B



Shred memory servers

Executable on various multicores

Hitachi, Renesas, NEC, Fujitsu, Toshiba, Denso, Olympus, Mitsubishi, Esol, Cats, Gaio, 3 univ.

OSCAR: Optimally Scheduled Advanced Multiprocessor
API : Application Program Interface

中間コード生成系

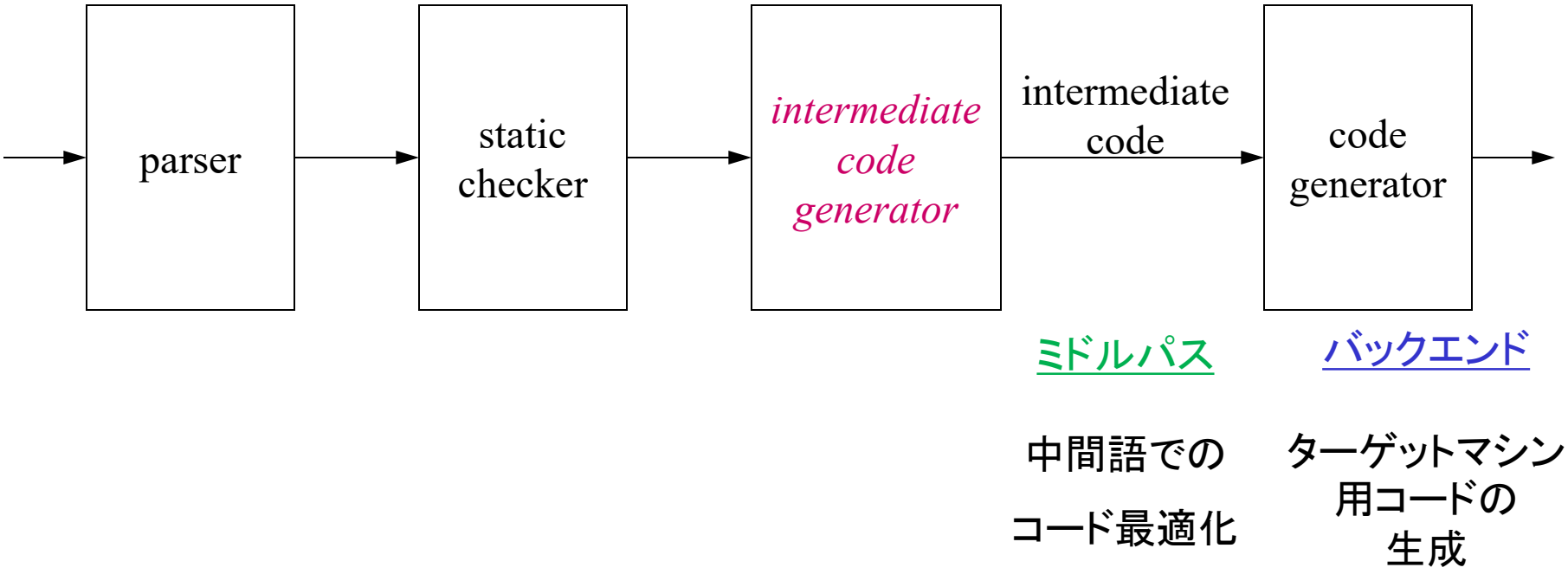
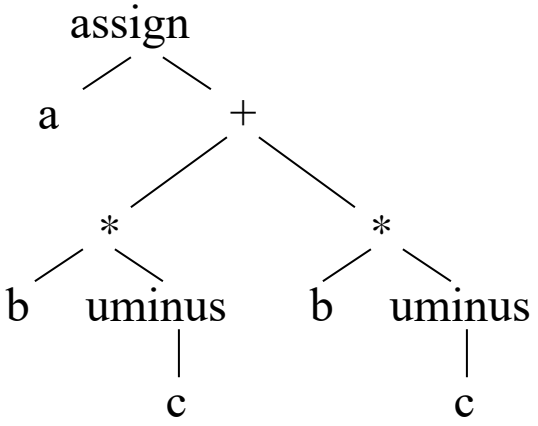


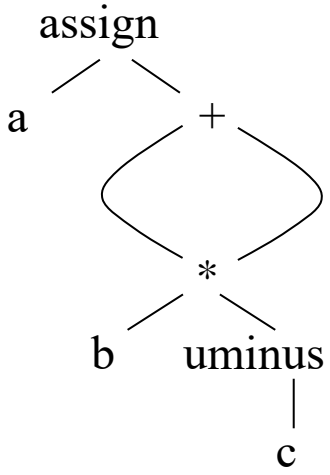
Fig. 8.1. Position of intermediate code generator.

<最適化の例>中間コード:3アドレスコード(四つ組)

Fig. 8.2. Graphical representations of a := b * -c + b * -c



(a) Syntax tree (構文木)
プログラムの階層構造を表す



(b) DAG : Directed Acyclic Graph

$t_1 := -c$
$t_2 := b * t_1$
$t_3 := -c$
$t_4 := b * t_3$
$t_5 := t_2 + t_4$
$a := t_5$

6文
(a) Code for the syntax tree.
最適化前のコード

$t_1 := -c$
$t_2 := b * t_1$
$t_5 := t_2 + t_2$
$a := t_5$

4文: 33%の高速化可能

(b) Code for the DAG.
最適化後のコード例

中間コード
中間語で記述された。一種の抽象機械に対するプログラム。

中間語:3アドレスコード(四つ組)
演算子、
ソースオペランド1、
ソースオペランド2、
デスティネーションオペランド

Fig. 8.5. Three address code corresponding to the tree and dag in Fig. 8.2.

コンパイラによる最適化

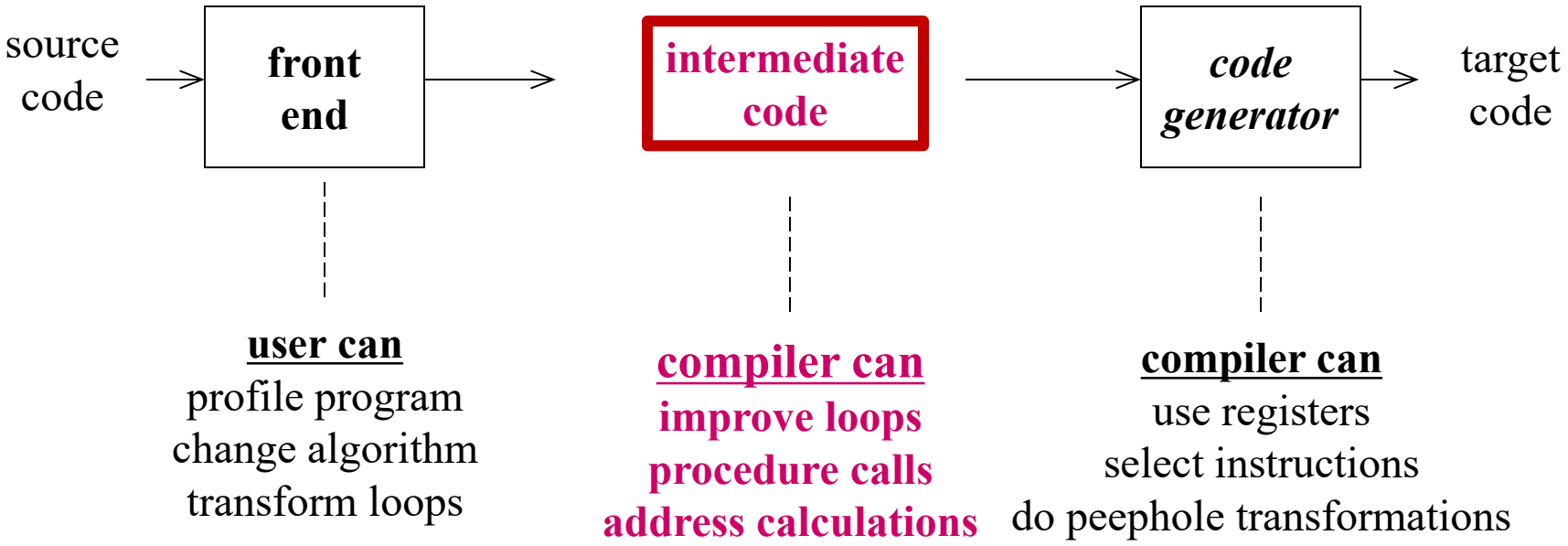


Fig. 10.1. Places for potential improvements by the user and the compiler.

Basic Blocks （基本ブロック）

基本ブロック:

- 分岐等がなく連続的に実行される一連の文の集合
- （条件）分岐文があるとその文が、基本ブロックの最後の文となる
- 分岐先ラベルを持つ文があると、その文が基本ブロックの最初の文になる。

$$
\begin{aligned}
t_1 &:= a * a \\
t_2 &:= a * b \\
t_3 &:= 2 * t_3 \\
t_4 &:= t_1 + t_3 \\
t_5 &:= b * b \\
a &:= t_4 + t_5
\end{aligned}
\qquad (9.1)
$$

3アドレスステートメント（文） **x:=y+z** において

“ **xを定義(define)する**“,

“**yとzを使用(use)する**,あるいは**参照(refer)する**“ という。

(Control) Flow Graph

有向グラフにより、基本ブロック間の制御フロー（プログラムの上から下、あるいは条件分岐等による実行順序）情報を表したもの

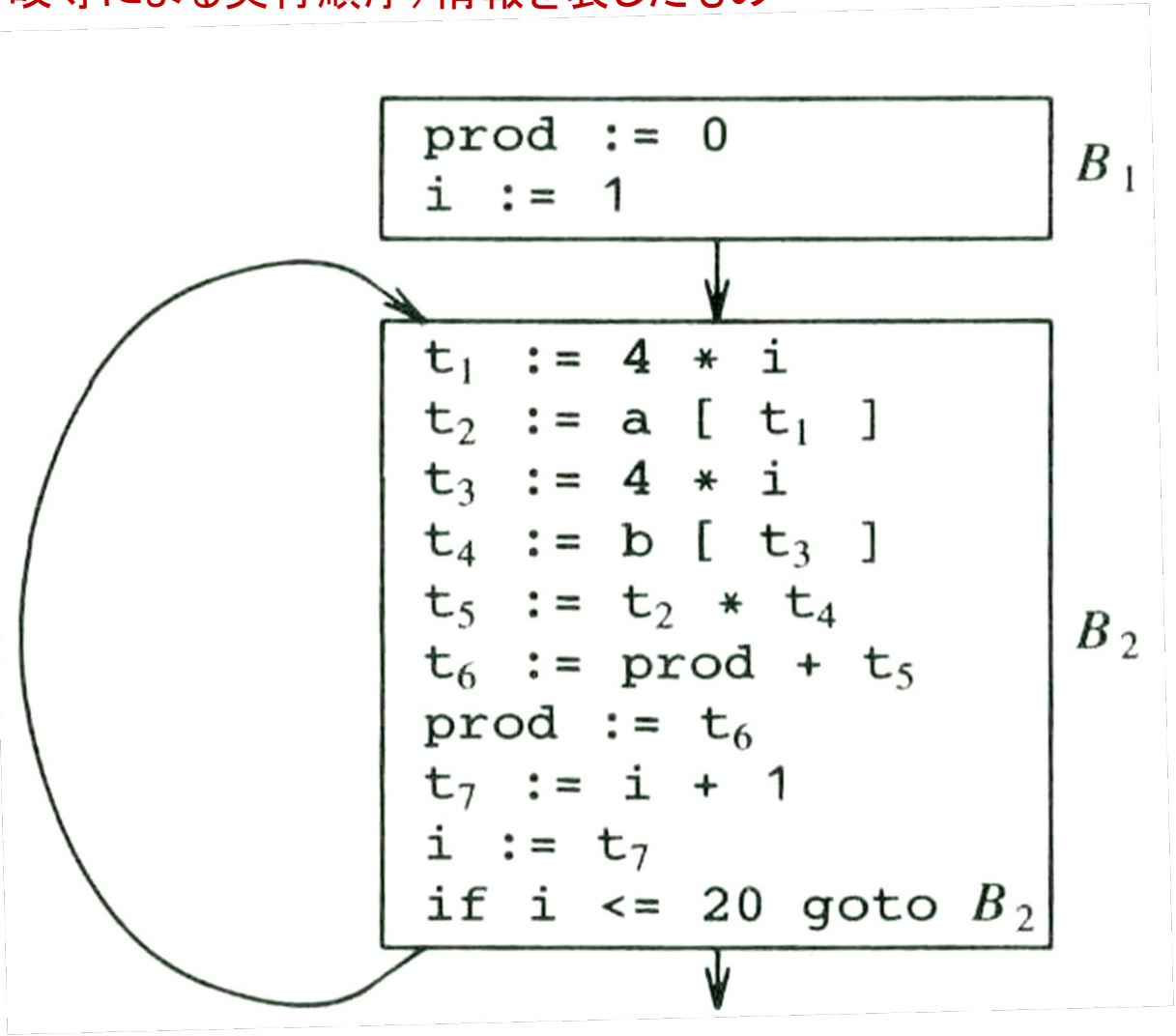


Fig. 9.9. Flow graph for program.

ノード：基本ブロック

・ある基本ブロックB_iの次に実行される、あるいは分岐により実行される可能性のある基本ブロックB_jには、ブロックB_iからB_jへの有向エッジが付けられる。

•ある基本ブロックの最後の文から、他あるいは自基本ブロックの最初の文への条件分岐、無条件ジャンプがある場合も有向エッジが付けられる。

•B_1直後にB_2が実行される場合

•B_1はB_2の先行ノード Predecessor と呼ばれる

•B_2はB_1の後続ノード Successor と呼ばれる

Flow graphの例

```
B1  (1)   i := m-1
    (2)   j := n
    (3)   t1 := 4*n
    (4)   v := a[t1]
B2  (5)   i := i+1
    (6)   t2 := 4*i
    (7)   t3 := a[t2]
    (8)   if t3 < v goto (5)
B3  (9)   j := j-1
    (10)  t4 := 4*j
    (11)  t5 := a[t4]
    (12)  if t5 > v goto (9)
B4  (13)  if i >= j goto (23)
B5  (14)  t6 := 4*i
    (15)  x := a[t6]
    (16)  t7 := 4*i
    (17)  t8 := 4*j
    (18)  t9 := a[t8]
    (19)  a[t7] := t9
    (20)  t10 := 4*j
    (21)  a[t10] := x
    (22)  goto (5)
B6  (23)  t11 := 4*i
    (24)  x := a[t11]
    (25)  t12 := 4*i
    (26)  t13 := 4*n
    (27)  t14 := a[t13]
    (28)  a[t12] := t14
    (29)  t15 := 4*n
    (30)  a[t15] := x
```

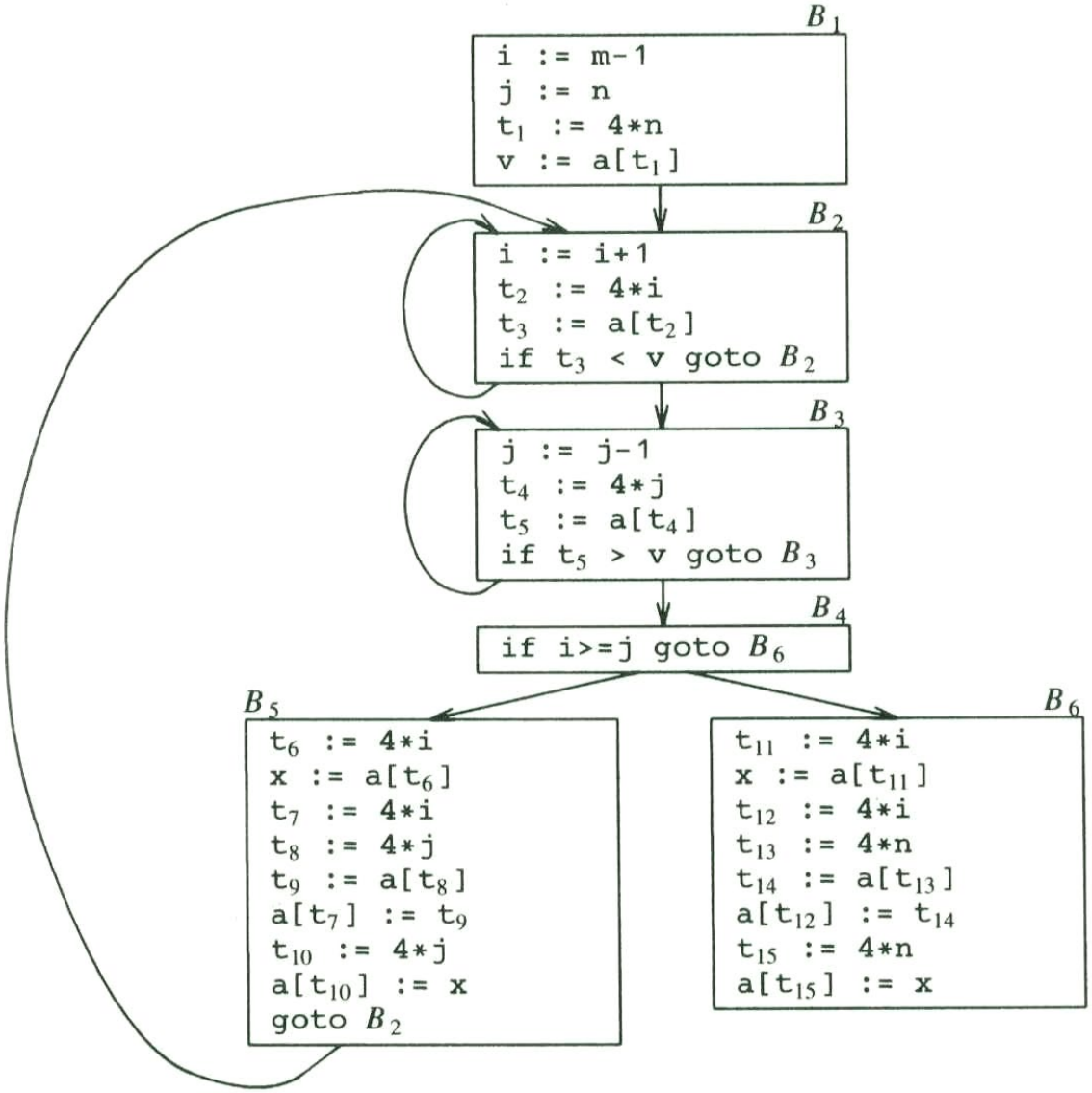
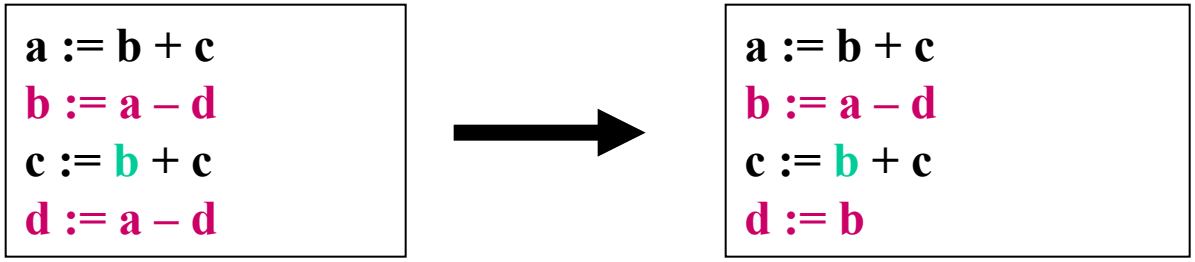


Fig. 10.5. Flow graph.

Structure-Preserving Transformations
プログラム構造を保った変換（最適化）

1. *Common subexpression elimination.（共通部分式の削除）*



2. *Dead-code elimination（死んだコードの除去）.* 基本ブロック中で文x:=y+zが出現した段階で、 xが“死んだ(dead)変数”、即ちそれ以降のプログラムで使用されない変数であれば、その文は安全に削除できる。

- 変数xが、その後の文の実行で使用される時、“生きている”、という。

3. *Renaming temporary variables（一時変数のリネーム）.* tが一時変数である文t:=b+cが存在する場合、tを他の一時変数uに置き換え、u:=b+cとし、その文以降のtの使用を全てuに置き換えても計算結果は変わらない。 一般的に、基本ブロックは、常に新しい一時変数を定義する形の基本ブロックに置き換えることができ、このような基本ブロックは、 “Normal Form Block” と呼ばれる。

4. *Interchange of statements（文の相互入れ替え）.*

$t_1 := b + c$
$t_2 := x + y$

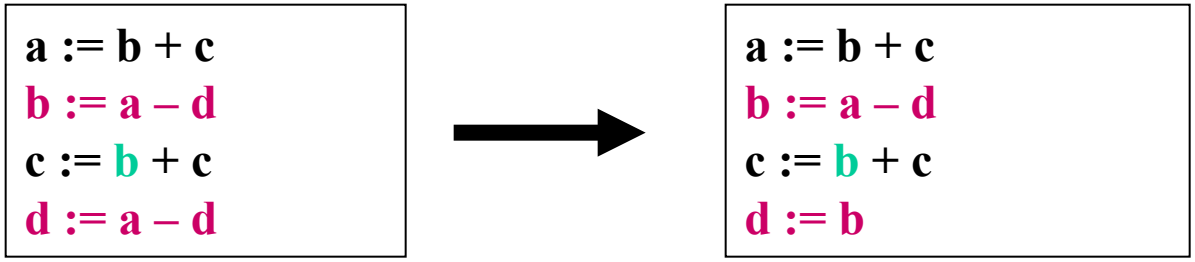
xとyの部分にt_1が現れなければ“定義後の利用”のデータ依存（フロー依存）、bとcの部分にt_2 が現れなければ“利用後の定義”（逆依存）、というデータ依存関係が二つの文間存在しないため、2文の計算順序を入れ替えても、2文を同時に実行（並列実行）しても結果は変わらない

代数的変換

- 式の集まりを代数的に等価で、高速に実行できる式に変換する。
  - 次のような式の基本ブロックからの削除
    - x:=x+0
    - x:=x*1
  - 関数呼び出しにより実行される計算コストの大きいべき乗のような処理を、等価でコストの低い演算に置き換える
    - x:=y**2　　を　　x:=y*y

Structure-Preserving Transformations
プログラム構造を保った変換（最適化）

1. *Common subexpression elimination.（共通部分式の削除）*



2. *Dead-code elimination（死んだコードの除去）.* 基本ブロック中で文x:=y+zが出現した段階で、 xが“死んだ(dead)変数”、即ちそれ以降のプログラムで使用されない変数であれば、その文は安全に削除できる。

- 変数xが、その後の文の実行で使用される時、“生きている”、という。

3. *Renaming temporary variables（一時変数のリネーム）.* tが一時変数である文t:=b+cが存在する場合、tを他の一時変数uに置き換え、u:=b+cとし、その文以降のtの使用を全てuに置き換えても計算結果は変わらない。 一般的に、基本ブロックは、常に新しい一時変数を定義する形の基本ブロックに置き換えることができ、このような基本ブロックは、 “Normal Form Block”と呼ばれる。

4. *Interchange of statements（文の相互入れ替え）.*

t_1 := b + c
t_2 := x + y

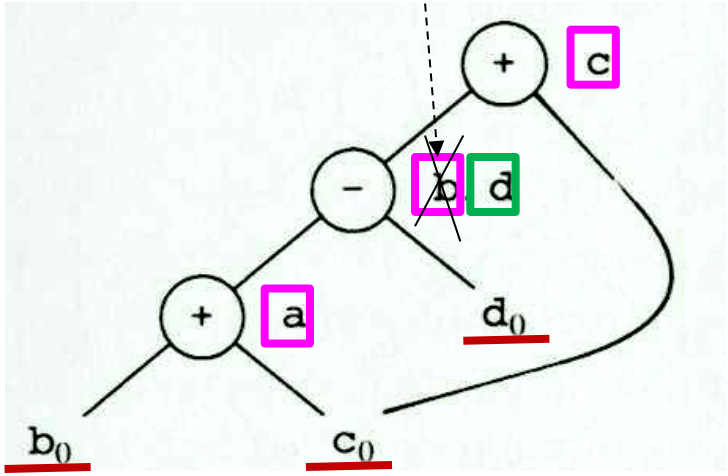
xとyの部分にt_1が現れなければ“定義後の利用”（フロー依存）、bとcの部分にt_2 が現れなければ“利用後の定義”（逆依存）、というデータ依存関係が二つの文間存在しないため、2文の計算順序を入れ替えても、2文を同時に実行（並列実行）しても結果は変わらない

DAGを用いた共通部分式の削除法

DAG（閉路の無い有向グラフ： Directed Acyclic Graph）

a := b + c
b := a – d
c := b + c
d := a - d

bがこれ以降使用されない、即ち死んでいる場合



a := b + c
d := a – d
c := d + c

bが生きている場合には、b:=d を追加するかこれ以降のプログラムで現れるbをdにリネームする

基本ブロック内での共通部分式削除に使う、DAGの作り方

- 葉のラベルは、定義前に使用される識別子(即ち変数名または定数)とし下添字$_0$を付ける
- 内部ノードのラベルは演算記号で、各ノードの右側に、そのノードで定義される識別子を書く。
- 新しいノード(節点)m をDAGに付け加える時に、同じ演算子で、同じ順序で同じ子を持つノードnが存在するならば、ノードmは付け加えずに、ノードnの右横に、ノードmで定義する識別子(左図d)を書き加え、中間コード生成時にはmにより定義される識別子名(左図の場合d)を用いる。
- ただし、左例で、b が基本ブロックの出口で生きている場合には、bにもdの値をコピーするか、あるいは以降のbをdにリネームする。

共通部分式： ある式Eと同じ計算がそれ以前に行われ、Eを計算するまでの間に、Eの右辺の変数の値が変更されていなければ、Eは共通部分式となる

Local common subexpression elimination

共通部分式: ある式Eと同じ計算がそれ以前に行われ、Eを計算するまでの間に、Eの右辺の変数の値が変更されていなければ、Eは共通部分式となる

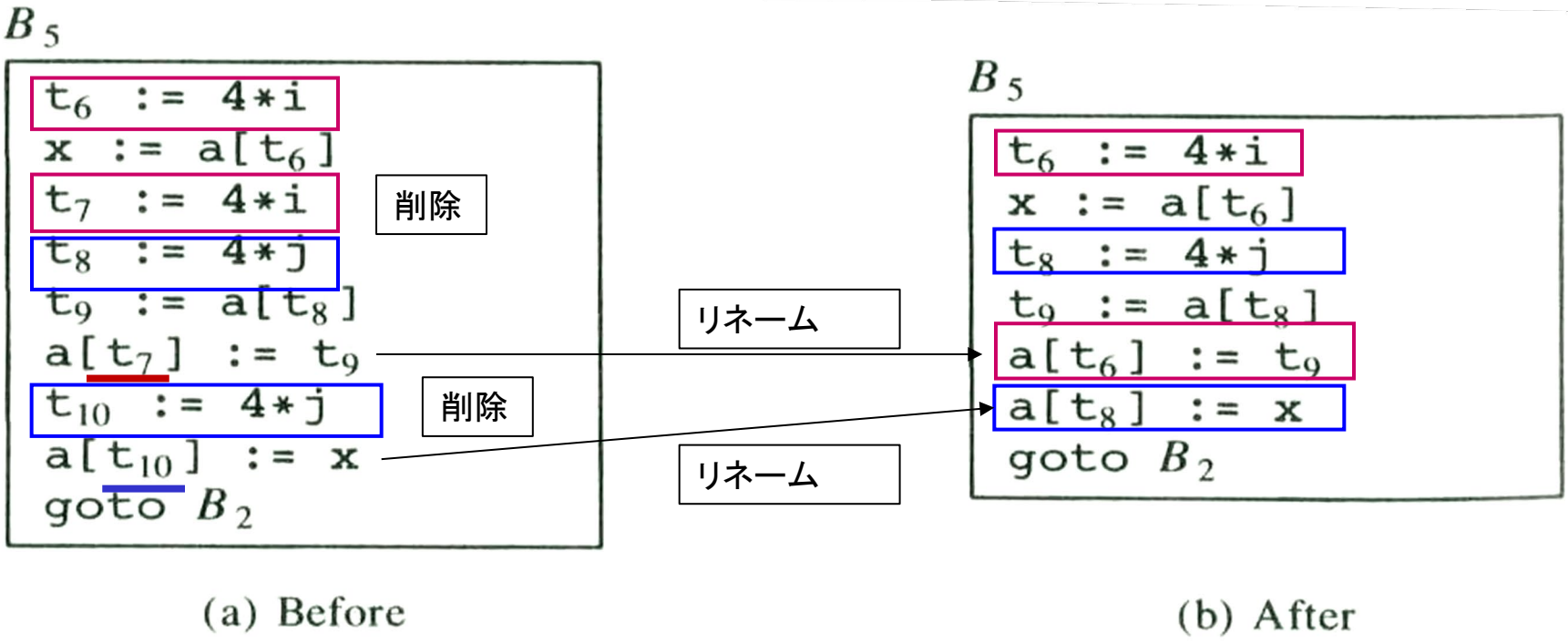
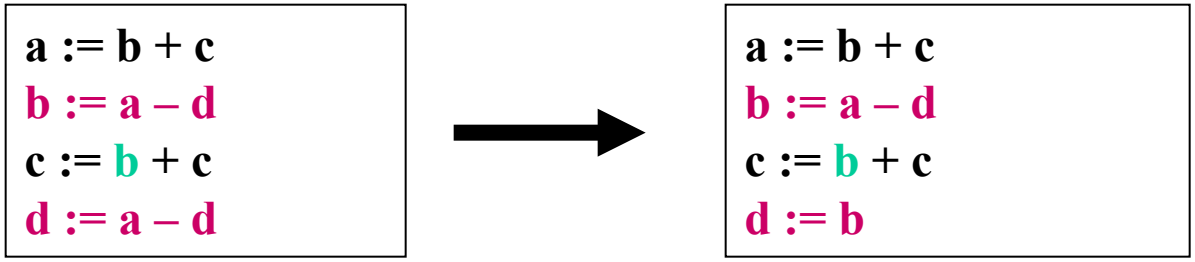


Fig. 10.6. Local common subexpression elimination.

Structure-Preserving Transformations
プログラム構造を保った変換（最適化）

1. *Common subexpression elimination.（共通部分式の削除）*



2. *Dead-code elimination（死んだコードの除去）.* 基本ブロック中で文x:=y+zが出現した段階で、 xが“死んだ(dead)変数”、即ちそれ以降のプログラムで使用されない変数であれば、その文は安全に削除できる。

- 変数xが、その後の文の実行で使用される時、“生きている”、という。

3. *Renaming temporary variables（一時変数のリネーム）.* tが一時変数である文t:=b+cが存在する場合、tを他の一時変数uに置き換え、u:=b+cとし、その文以降のtの使用を全てuに置き換えても計算結果は変わらない。 一般的に、基本ブロックは、常に新しい一時変数を定義する形の基本ブロックに置き換えることができ、このような基本ブロックは、 “Normal Form Block”と呼ばれる。

4. *Interchange of statements（文の相互入れ替え）.*

t_1 := b + c
t_2 := x + y

xとyの部分にt_1が現れなければ“定義後の利用”（フロー依存）、bとcの部分にt_2 が現れなければ“利用後の定義”（逆依存）、というデータ依存関係が二つの文間存在しないため、2文の計算順序を入れ替えても、2文を同時に実行（並列実行）しても結果は変わらない

共通部分式では無い場合のDAGと、DAGを用いたDead Code Elimination

```
a := b + c
b := b - d
c := c + d
e := b + c
```

(10.4)

aが死んでいれば、生きていないaを根(root)にもつノードを削除して良い。

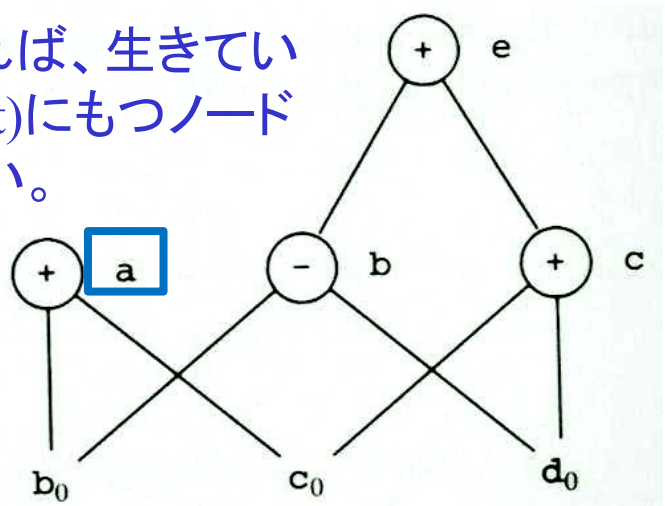


Fig. 10.12. Dag for basic block (10.4)

- 同じ b+c　でも bの値が更新されている場合には共通部分式の対象にならず、DAG も左図のようになる。

- DAGを用いるとDead Code Elimination (死んだコードの削除）も簡単に行える。

　生きていない変数をもつ根(root)ノードを削除する。

Fig. 9.15. Three-address code for block B_2.

(1) t_1 := 4 * i
(2) t_2 := a [t_1]
(3) t_3 := 4 * i
(4) t_4 := b [t_3]
(5) t_5 := t_2 * t_4
(6) t_6 := prod + t_5
(7) prod := t_6
(8) t_7 := i + 1
(9) i := t_7
(10) if i <= 20 goto (1)

(1) t_1 := 4 * i
(2) t_2 := a [t_1]
(3) t_4 := b [t_1]
(4) t_5 := t_2 * t_4
(5) prod := prod + t_5
(6) i := i + 1
(7) if i <= 20 goto (1)

DAG（閉路の無い有向グラフ）

基本ブロック内の最適化によく用いるデータ構造

•葉のラベルは一意な識別子(Identifier),即ち変数名または定数

•内部ノードのラベルは演算記号

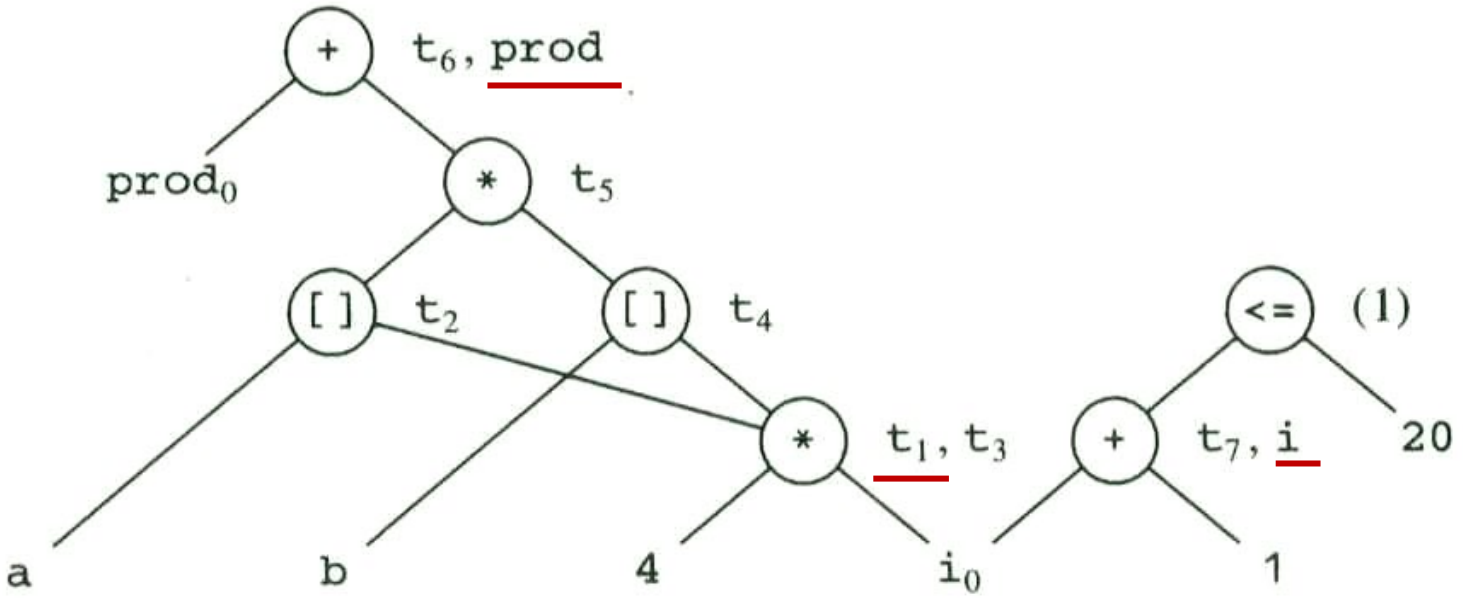
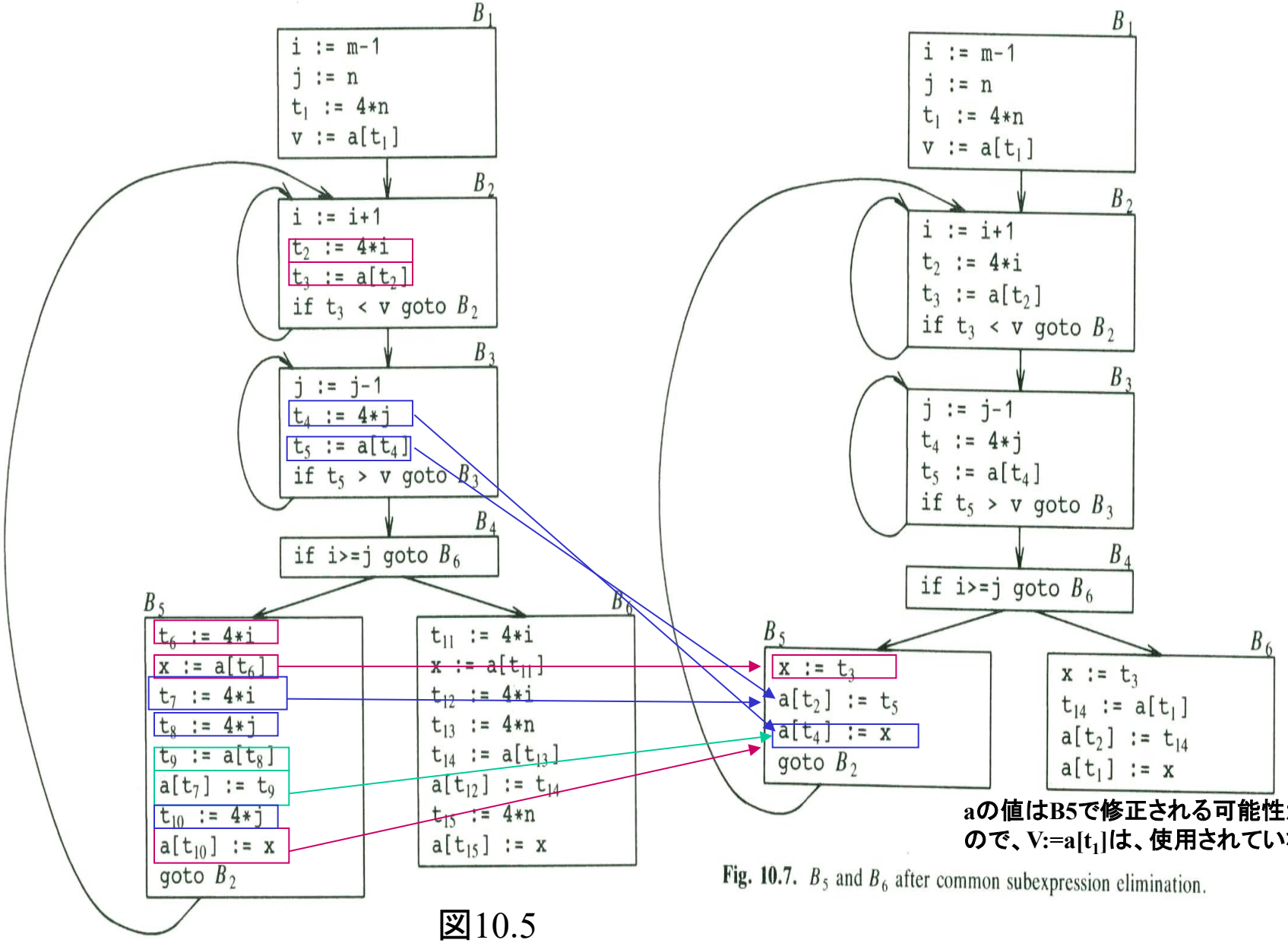


Fig. 9.16. Dag for block of Fig. 9.15.

複数の基本ブロックにまたがる最適化

B_5 and B_6 after common subexpression elimination



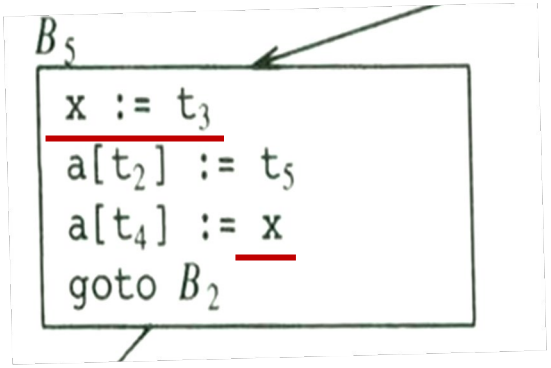
aの値はB5で修正される可能性があるので、V:=a[t_1]は、使用されていない。

Fig. 10.7. B_5 and B_6 after common subexpression elimination.

図10.5

Copy Propagation (複写伝播)

- コピー文　f:=g の文が出現したら、以降の文で、変数 f を　gに置き換える Copy Propagation (複写伝播) を行うと、変数への代入を除去できる可能性が増える。



B_5のCopy Propagation

x=:t_3

a[t_2]=:t_5

a[t_4] =:t_3

goto B_4

x:=2
y:=3
・
・
z:=60/(x+y)
if z <= 20 goto (1)

定数伝播
・
・
z:=60/(2+3)
if z <= 20 goto (1)

たたみ込み
コンパイル時計算
・プログラム変換
・
・
goto (1)

無用コードの削除

- 変数xがその後の実行で使用される時生きているという。使用されないとき死んでいる(dead)
- 文あるいはコードが計算する値がそれ以降使用されない場合死んでいる、あるいは無用という。
- 定数をコンパイル時に計算して、式を定数に置き換える定数計算（定数のたたみ込み）により無用コードが出現する。

＜演習＞

c:=b+c
u:=b*c
s:=b+c
e:=c+b
a:=b*c
の計算の
1)　DAGを書く（ただし、uとeは基本ブロックの出口で死んでいるとする）
（DAGは、紙に手書きしたものをスキャンして、 Wordファイルに貼ってもOKです。）

2) DAGを用いて共通部分式を削除したコード(式）を書く